SEMANA 15 · PATRONES DE INGENIERÍA DE DATOS

Datos como activo: de la fuente a la decisión

Parte 1 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 15 · Periodo 2026-2 · Barranquilla, Colombia

Las cinco V, los tipos de datos y el pipeline que convierte datos en decisiones.

ENCUADRE

Una empresa con datos, pero sin visión de conjunto

La Corporación Industrial del Caribe genera datos en seis lugares distintos y ninguno conversa con el otro.



Fuente: caso transversal del curso; íconos AWS Architecture Icons

Del dato crudo a la decisión

Un dato sin contexto no sirve; el valor aparece cuando se convierte en una acción.

cada paso agrega valor y reduce el volumen

Dato

el hecho crudo



87,4
lectura del termopar
horno 3, 14:02 h

+ contexto

Información

dato con contexto



Temperatura media
del turno A: 87 °C
23 lecturas válidas

+ patrones

Conocimiento

patrón que se repite



Tras 6 horas de marcha,
el horno 3 sube 9 °C
sobre su media

+ acción

Decisión

acción con impacto

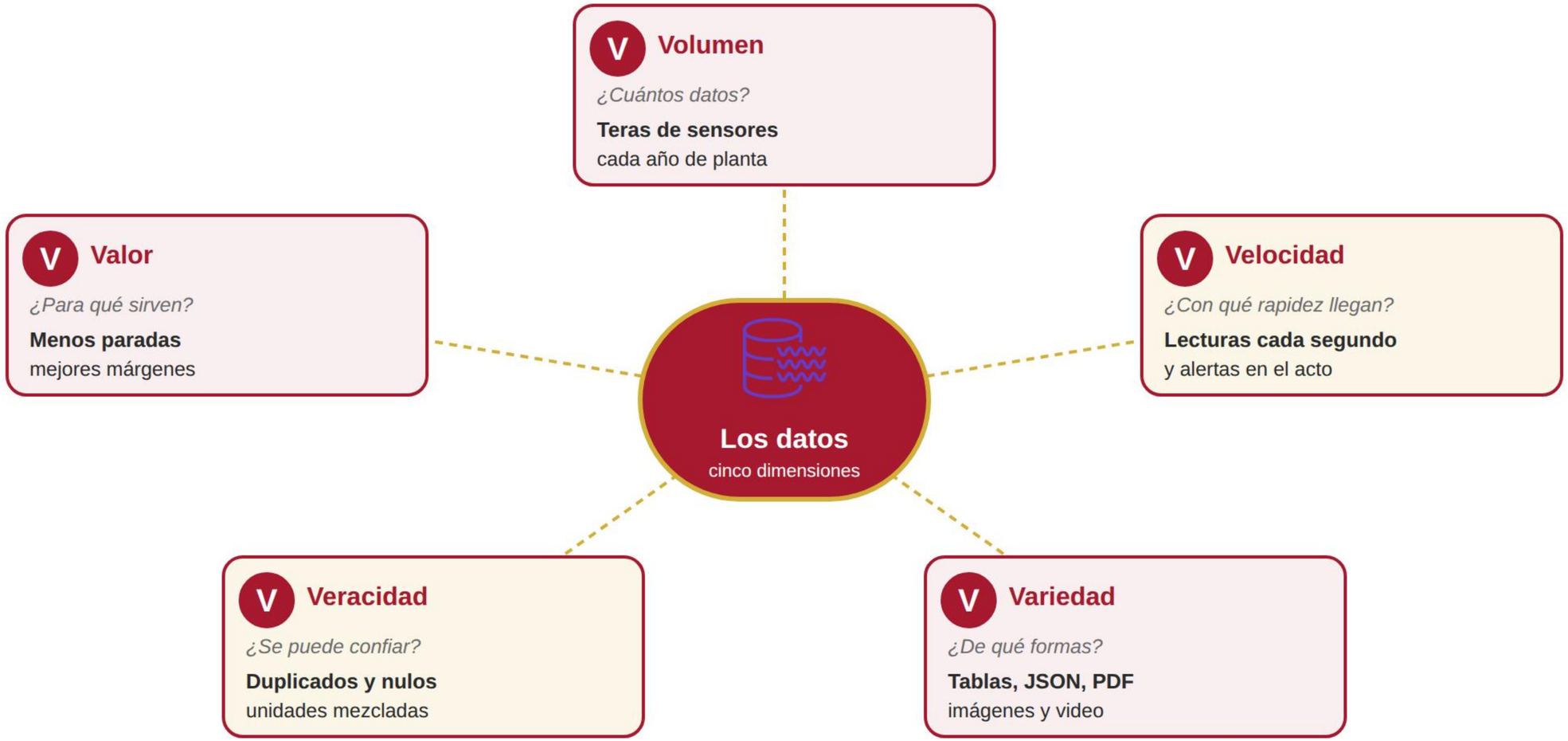


Mantenimiento del
horno 3 antes del
turno nocturno

Fuente: adaptado de la jerarquía DIKW (Ackoff, 1989); ejemplo propio del caso CIC

Las cinco V: cinco preguntas sobre cualquier dato

Volumen, velocidad, variedad, veracidad y valor describen el reto de datos de una empresa.



Fuente: modelo de las V del big data (Laney, 2001; ampliado por IBM y AWS)

Volumen: de megas a petabytes en la misma empresa

Cada fuente crece a un ritmo distinto, y el almacenamiento tiene que crecer sin rediseñarse.



Escala logarítmica: cada marca vale 1.000 veces la anterior · cifras ilustrativas

Fuente: cifras ilustrativas del caso CIC; concepto de volumen según AWS Big Data

Velocidad: qué tan frescos deben estar los datos

Cuanto más rápida la decisión, más cerca del tiempo real debe estar el procesamiento.

¿qué tan frescos necesitamos los datos?

Mensual	Diario	Cada hora	Segundos	Milisegundos
				
Reporte contable al cierre del mes	Carga nocturna del ERP al almacén	Tablero de producción por turno	Alerta de temperatura del horno	Visión artificial rechaza el defecto
datos con semanas de edad	**hasta 24 h de retraso**	**hasta 1 h de retraso**	**casi tiempo real**	**tiempo real estricto**

LOTES (batch)

los datos se acumulan y se procesan juntos

FLUJO (streaming)

se procesa cada evento al llegar

Fuente: adaptado de AWS Big Data Blog — batch vs streaming

Variedad: tres formas de guardar la misma realidad

La mayoría de los datos nuevos de una empresa no caben en una tabla.

Estructurados

filas y columnas fijas

Pedido	Cliente	Fecha	Total
1021	Cartón SA	03/09	1.2 M
1022	Caribe Pack	03/09	860 K
1023	Agro Sur	04/09	2.4 M
1024	Cartón SA	05/09	530 K
1025	Plas Norte	05/09	970 K

Tablas del ERP

Amazon RDS · Aurora · Redshift

esquema antes de guardar

Semiestructurados

forma flexible, con etiquetas

```
{
  "sensor": "H3-T01",
  "ts": "2026-09-03T14:02",
  "temp": 87.4,
  "alarma": false,
  "etiquetas": ["horno", "planta1"]
}
```

Eventos JSON de sensores

DynamoDB · DocumentDB · S3

esquema flexible, se lee al consultar

No estructurados

sin esquema definido

foto | PDF | video

`defecto_linea2_0903.jpg`

`factura_proveedor_88421.pdf`

el contenido no cabe en columnas

Fotos, PDF y video

Amazon S3

sin esquema; hay que extraer significado

Fuente: adaptado de AWS — tipos de datos para analítica; íconos AWS Architecture Icons

Veracidad: un dato dudoso es peor que no tener dato

Los errores de calidad se detectan con reglas automáticas, no con la intuición del analista.

Lecturas crudas que llegan del historiador

sensor	fecha	temp	unidad
H3-T01	03/09 14:02	87,4	°C
H3-T01	03/09 14:02	87,4	°C
H3-T02	03/09 14:03	(vacío)	°C
H3-T01	31/02/2026	88,1	°C
H3-T04	03/09 14:05	188,6	°F
H3-T05	03/09 14:06	4000	°C
H3-T06	03/09 14:07	86,9	°C

- duplicado
- valor nulo
- fecha imposible
- unidad distinta
- valor atípico

Reglas de calidad (AWS Glue Data Quality)

Completitud	sin vacíos obligatorios
Validez	formatos y rangos correctos
Unicidad	sin registros repetidos
Consistencia	una sola unidad
Oportunidad	llegan a tiempo

Cuarentena

lo que falla se aparta y se avisa

nunca se borra en silencio

Zona curada

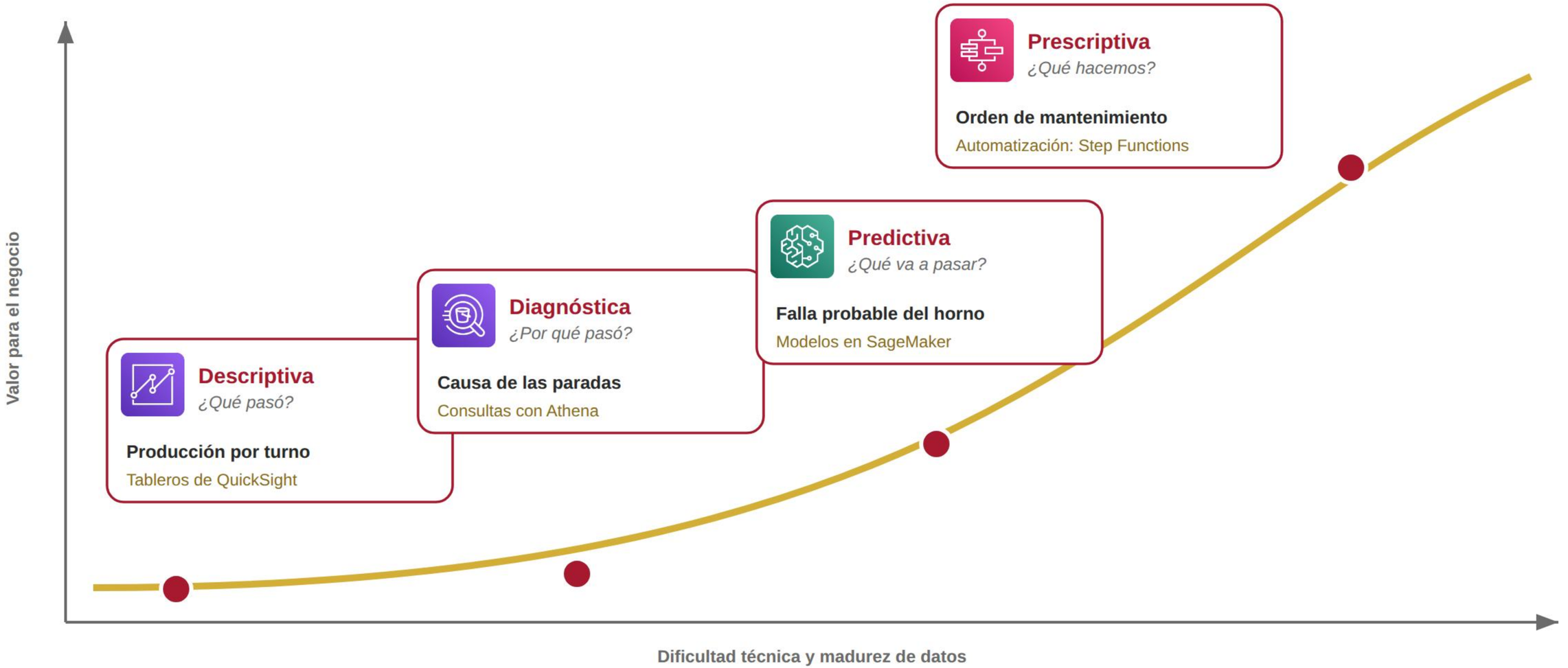
solo datos validados llegan al análisis

y a los tableros de gerencia

Fuente: adaptado de AWS Glue Data Quality (documentación AWS); ejemplo propio

Valor: cuatro niveles de analítica, cada uno más valioso

Se empieza describiendo lo que pasó y se avanza hacia automatizar lo que se debe hacer.

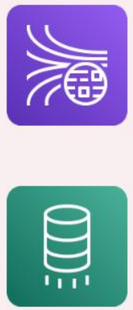


Fuente: adaptado de Gartner — modelo de madurez analítica; servicios AWS

El pipeline de datos: cinco etapas, un mismo camino

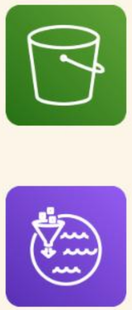
Ingestar, almacenar, procesar, analizar y visualizar es el mapa de toda la semana.

1 Ingestar
traer los datos



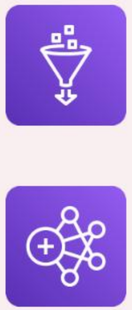
Kinesis · DMS

2 Almacenar
guardarlos a bajo costo



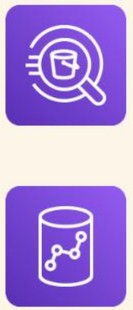
S3 · Lake Formation

3 Procesar
limpiar y transformar



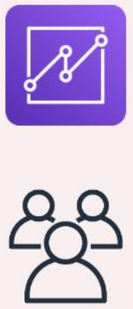
Glue · EMR

4 Analizar
hacer preguntas



Athena · Redshift

5 Visualizar
mostrar y decidir



QuickSight



Cifrado



Identidad



Auditoría



Monitoreo

Seguridad y gobierno en todas las etapas

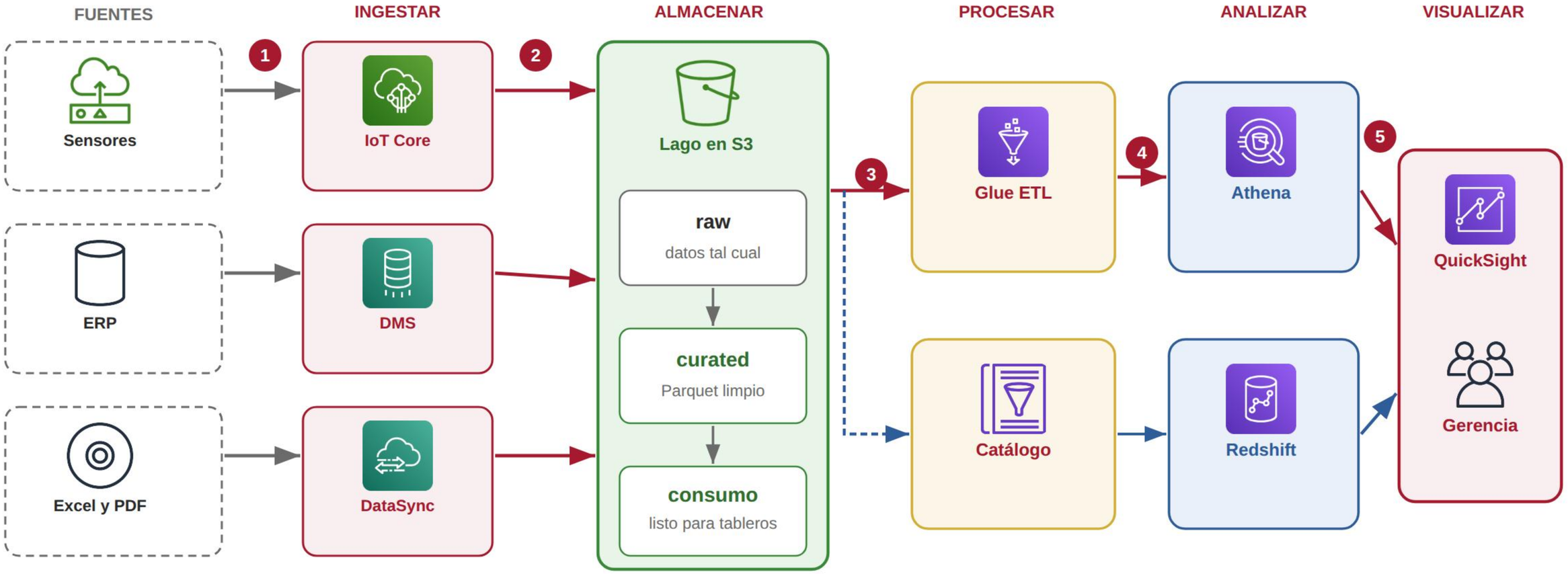
Datos en movimiento (etapas 1 y 3) → datos en reposo (etapa 2) → datos al servicio de personas (etapas 4 y 5)

Fuente: adaptado de AWS — Modern Data Architecture / Data Analytics Lens; íconos AWS

CASO APLICADO

El pipeline en la Corporación Industrial del Caribe

Tres fuentes distintas convergen en un solo lago y alimentan los tableros de gerencia.



1 las fuentes entran 2 aterrizan crudas 3 Glue limpia y cataloga 4 se consulta con SQL 5 se visualiza

Sensores con IoT Core, ERP con DMS y archivos con DataSync: tres fuentes, un solo lago.

Fuente: adaptado de AWS Architecture Center — Build a data lake; caso CIC

OLTP y OLAP: operar no es lo mismo que analizar

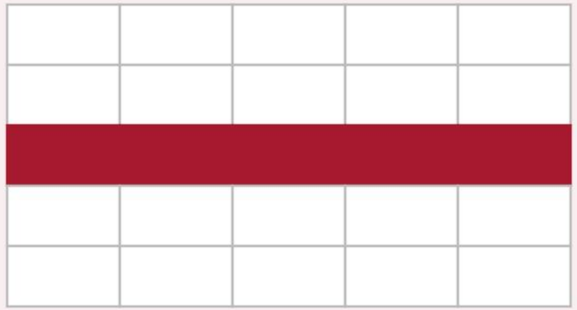
Una base transaccional y una analítica se diseñan distinto; por eso el pipeline las separa.

OLTP: operar el negocio



Muchas operaciones pequeñas y rápidas

insertar un pedido, descontar inventario



se lee y escribe la fila completa

esquema normalizado · datos actuales

milisegundos por operación

ETL / ELT

copiar sin frenar la operación

OLAP: analizar el negocio



Pocas consultas enormes sobre mucha historia

ventas por mes y por región en cinco años



se lee solo la columna que se necesita

esquema para análisis · historia completa

segundos o minutos por consulta

Fuente: adaptado de AWS Prescriptive Guidance — OLTP vs OLAP

PRINCIPIO DE DISEÑO

Un solo almacén, muchos motores de análisis

Separar almacenamiento y cómputo permite usar la herramienta adecuada sobre la misma copia de los datos.



ANTES: servidor y disco juntos

Equipo A — *copia 1* — Equipo B — *copia 2* — Equipo C

cada equipo con su copia; crecer obliga a comprar ambos

AHORA: almacenamiento y cómputo desacoplados

Cada motor se enciende cuando hace falta,
se paga mientras trabaja y se apaga.
Una sola copia de los datos para todos.

si aparece una herramienta nueva, se conecta al mismo lago

Fuente: adaptado de AWS Big Data Blog — decoupled storage and compute

Seguridad y gobierno desde el primer día

El dato es un activo sensible: se protege por capas y no se agrega al final.

Cumplimiento

Ley 1581 de 2012 de protección de datos personales (habeas data) y políticas de la empresa

Auditoría

Quién consultó qué dato y cuándo: AWS CloudTrail y registros de acceso

Descubrimiento de datos sensibles

Amazon Macie encuentra cédulas, correos y teléfonos dentro de S3

Permisos por tabla, columna y fila

IAM y AWS Lake Formation: cada rol ve solo lo que necesita

Cifrado

En reposo con AWS KMS y en tránsito con TLS; las llaves las controla la empresa

Fuente: adaptado de AWS Well-Architected — Data Analytics Lens y Ley 1581 de 2012

Lo que queda de la Parte 1

Los datos son un activo; el pipeline es el camino que lo convierte en decisiones.

1 Cinco V

volumen, velocidad, variedad, veracidad
y valor describen el reto

2 Tres formas

estructurados, semiestructurados
y no estructurados

3 Cinco etapas

ingestar, almacenar, procesar,
analizar y visualizar



Lo que sigue: la Parte 2 abre la puerta de entrada

¿Cómo llegan los datos a la nube: por lotes, en flujo, desde bases de datos y desde la planta?

Fuente: síntesis propia del curso; íconos AWS Architecture Icons

SEMANA 15 · PATRONES DE INGENIERÍA DE DATOS

Ingesta: llevar los datos a la nube

Parte 2 de 4 · Unidad 3

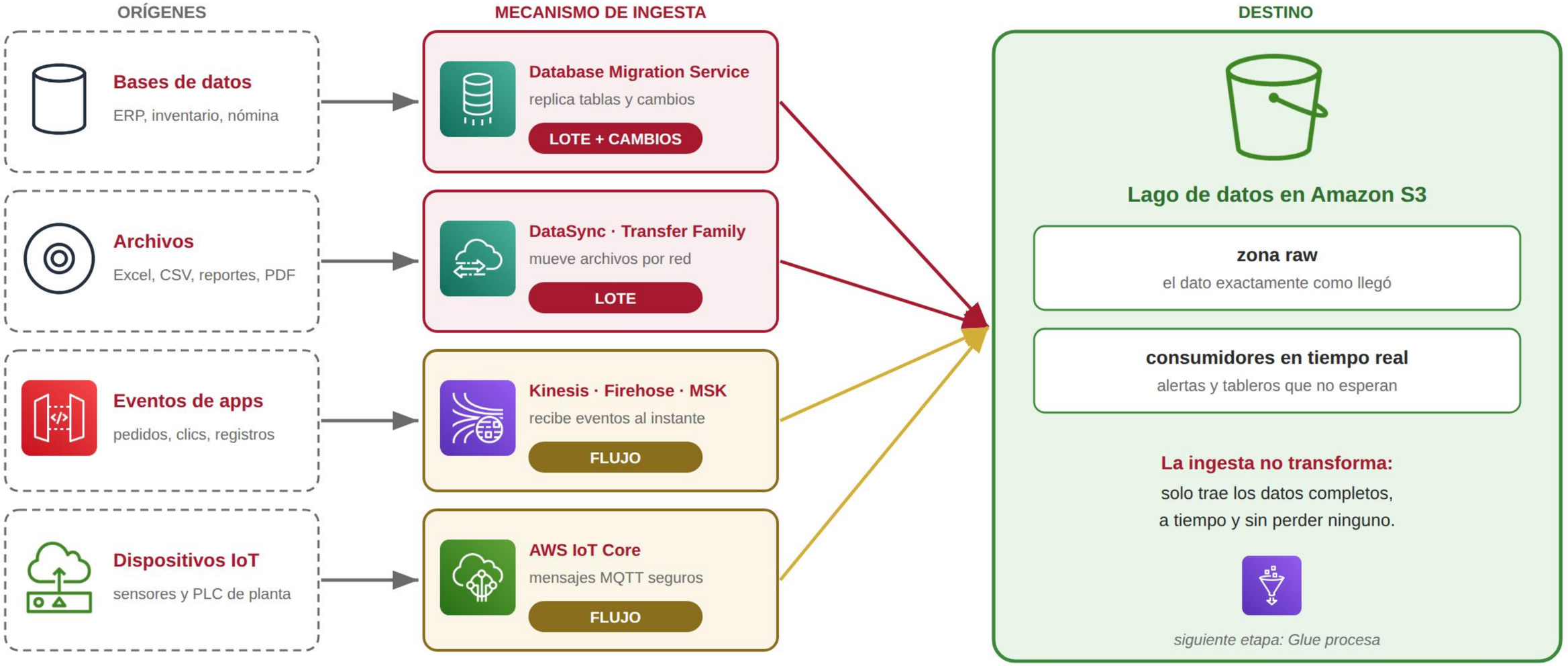
Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 15 · Periodo 2026-2 · Barranquilla, Colombia

Lotes y flujo, Kinesis, Kafka gestionado, replicación de bases de datos y sensores de planta.

ENCUADRE

La ingesta: la puerta de entrada del pipeline

Cada tipo de origen tiene su propio mecanismo, y casi todos terminan en el mismo lago.



Fuente: adaptado de AWS Big Data Blog — ingestion patterns; íconos AWS Architecture Icons

DOS RITMOS

Por lotes o en flujo: dos ritmos de ingesta

El lote acumula y descarga de una vez; el flujo mueve cada evento apenas ocurre.

POR LOTES (batch)

camión que descarga cada noche



Latencia
horas a días

Costo y complejidad
bajos: herramientas simples

Ideal para
cierres, informes, cargas históricas

EN FLUJO (streaming)

banda transportadora continua



Latencia
milisegundos a segundos

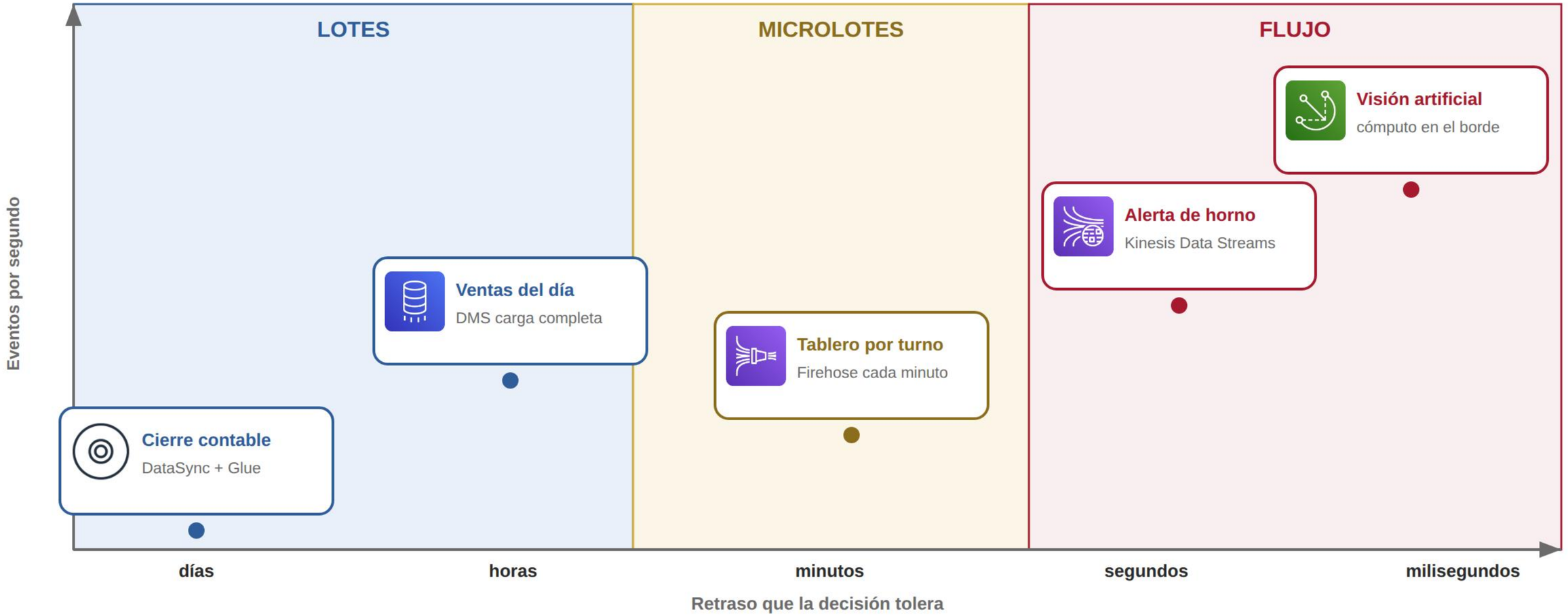
Costo y complejidad
mayores: siempre encendido

Ideal para
alertas, tableros vivos, IoT

Fuente: adaptado de AWS Well-Architected Data Analytics Lens — ingestion

Elegir el ritmo: qué retraso tolera la decisión

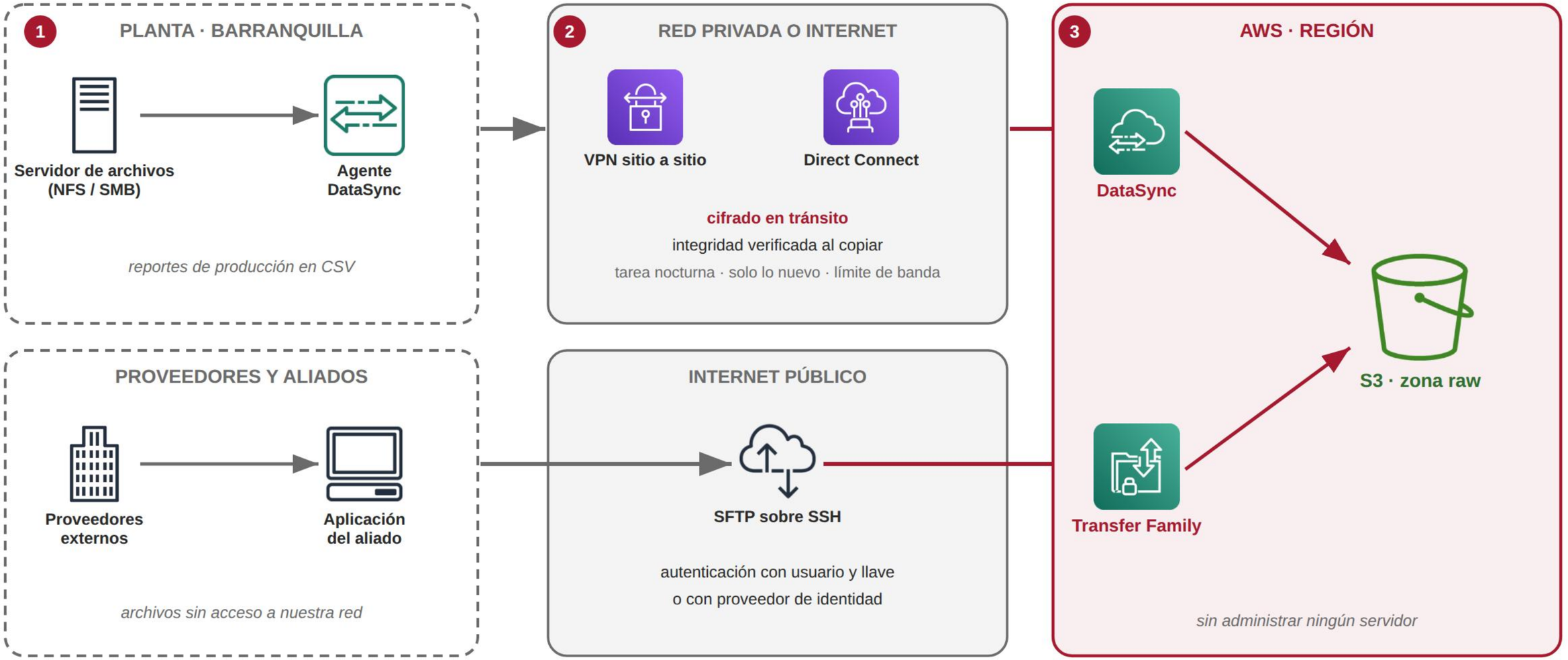
La pregunta correcta no es cuántos datos hay, sino cuánto puede esperar quien los necesita.



Fuente: adaptado de AWS Big Data Blog — choosing a streaming or batch approach; casos CIC

Mover archivos a la nube de forma confiable

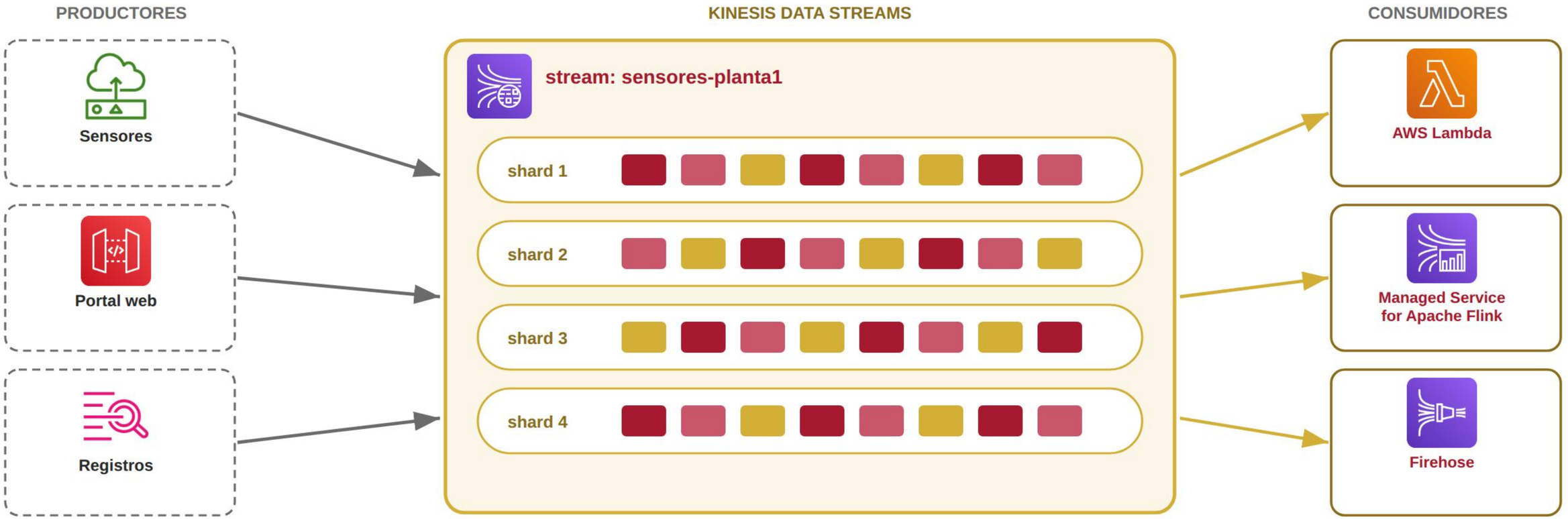
DataSync copia desde la planta y Transfer Family recibe archivos de aliados externos.



Fuente: adaptado de AWS DataSync y AWS Transfer Family (documentación AWS)

Kinesis Data Streams: anatomía de un flujo

Los datos se reparten en shards, se conservan un tiempo y varios consumidores los leen por separado.



Retención de datos
24 h por defecto, hasta 365 días

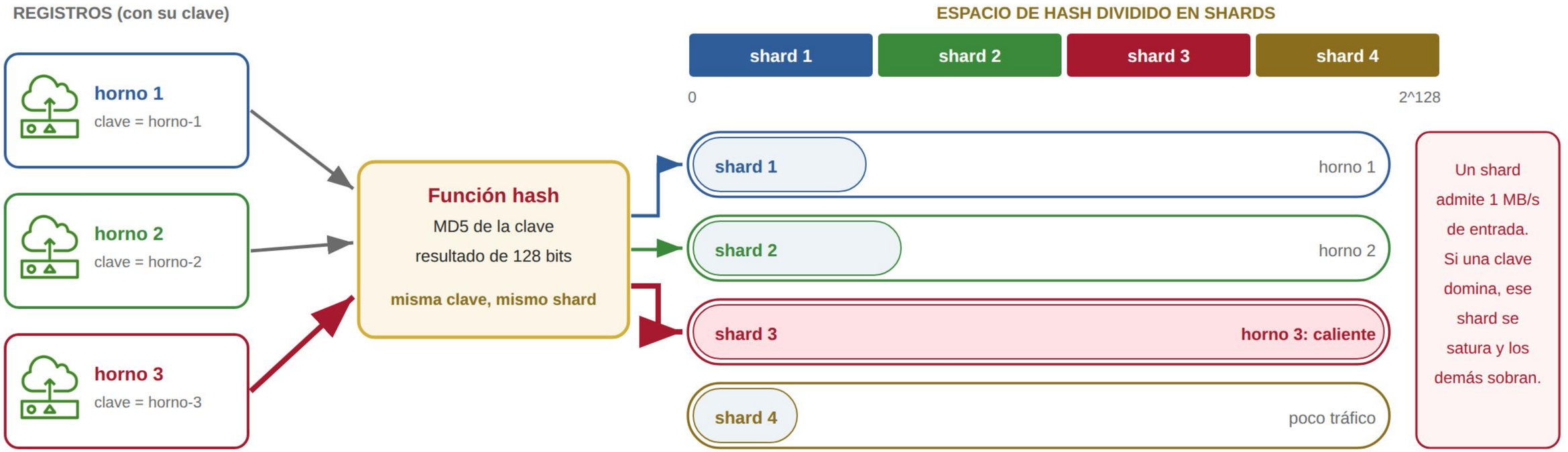
Capacidad por shard
entra 1 MB/s o 1.000 registros/s

Modos de capacidad
aprovisionado o bajo demanda

Fuente: adaptado de AWS — Amazon Kinesis Data Streams Developer Guide

Claves de partición: cómo se reparte la carga

La clave decide el shard: una clave bien repartida usa toda la capacidad del stream.



Lo que sí garantiza Kinesis

Orden de llegada dentro de cada shard.
Todos los registros de un mismo horno quedan juntos, en secuencia, para calcular tendencias correctamente.

El problema del shard caliente

Elegir una clave con poca variedad concentra el tráfico.
Solución: claves más repartidas, por ejemplo horno-3 más el número del sensor.

Fuente: adaptado de AWS — Kinesis Data Streams: partition keys and hot shards

Amazon Data Firehose: entrega gestionada al lago

Firehose junta, transforma y entrega los datos sin que administres servidores ni shards.



Sin shards que administrar · escala automática · casi tiempo real (según el buffer)

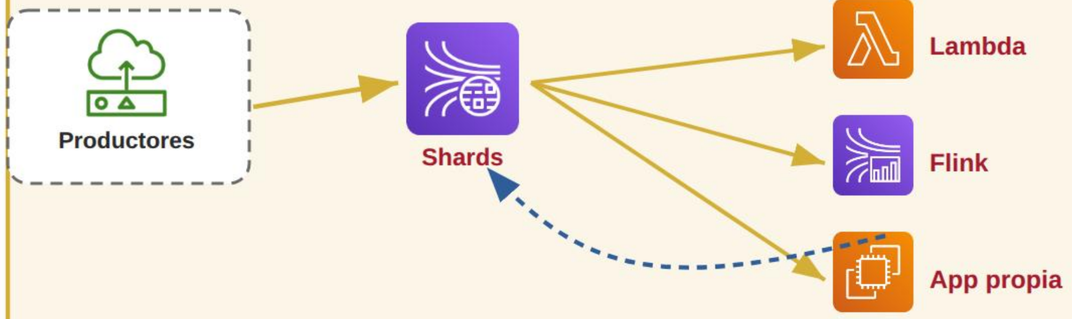
ideal cuando el destino final es el lago y no hace falta releer el flujo

Fuente: adaptado de AWS — Amazon Data Firehose Developer Guide

Streams frente a Firehose: control o comodidad

Data Streams da control fino sobre la lectura; Firehose entrega al destino sin administrar nada.

Kinesis Data Streams: tú decides cómo se lee



- milisegundos
- varios consumidores
- retención hasta 365 días
- tú administras la capacidad

Amazon Data Firehose: entrega gestionada



- sin shards ni capacidad que calcular
- casi tiempo real (buffer)
- sin releer: un solo destino por entrega

Se combinan: Data Streams para leer en vivo, Firehose para entregar el mismo flujo al lago

¿Cuál escojo?

¿Necesitas varios consumidores o releer datos?

Data Streams

¿Solo quieres llevar el flujo al lago o a Redshift?

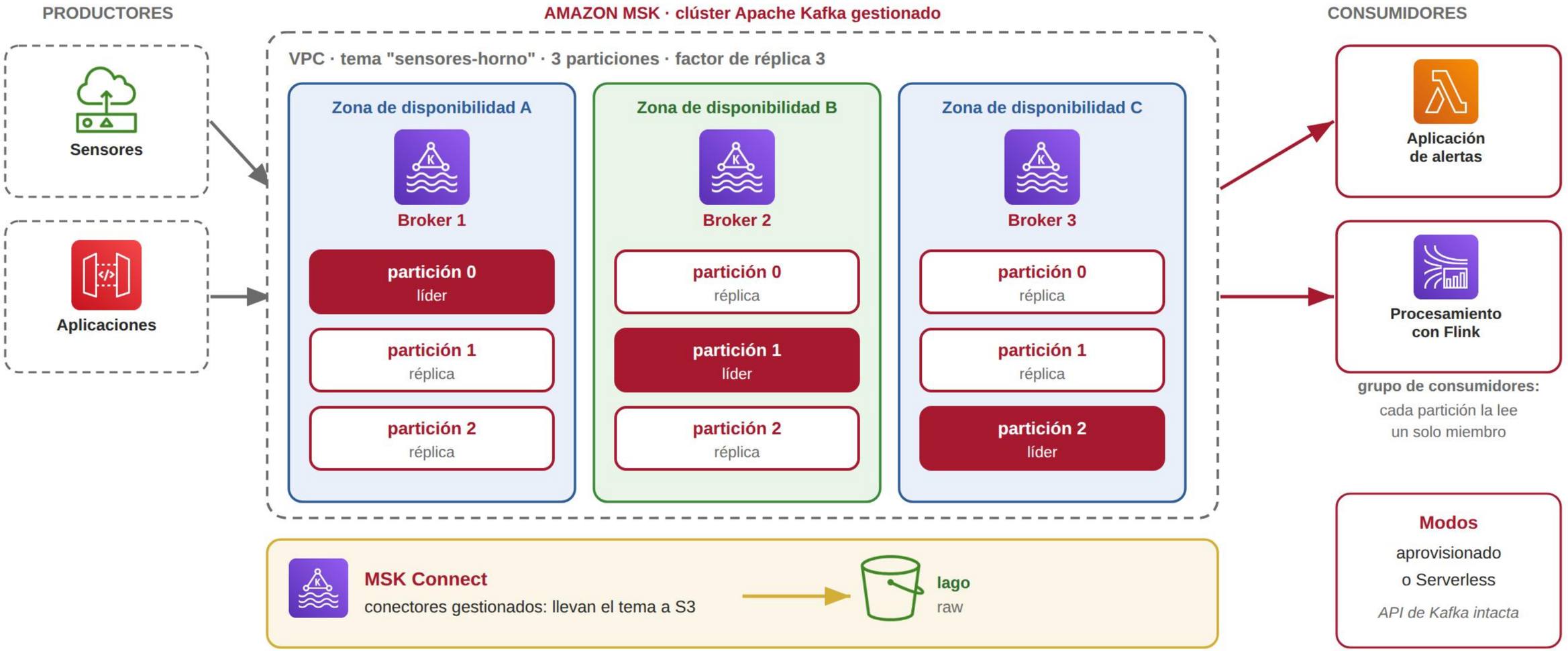
Firehose

¿Necesitas ambos? Streams en el centro y Firehose como un consumidor más

Fuente: adaptado de AWS — Choosing between Kinesis Data Streams and Firehose

Kafka gestionado: Amazon MSK

Es Apache Kafka estándar, con los brokers repartidos en tres zonas y operados por AWS.

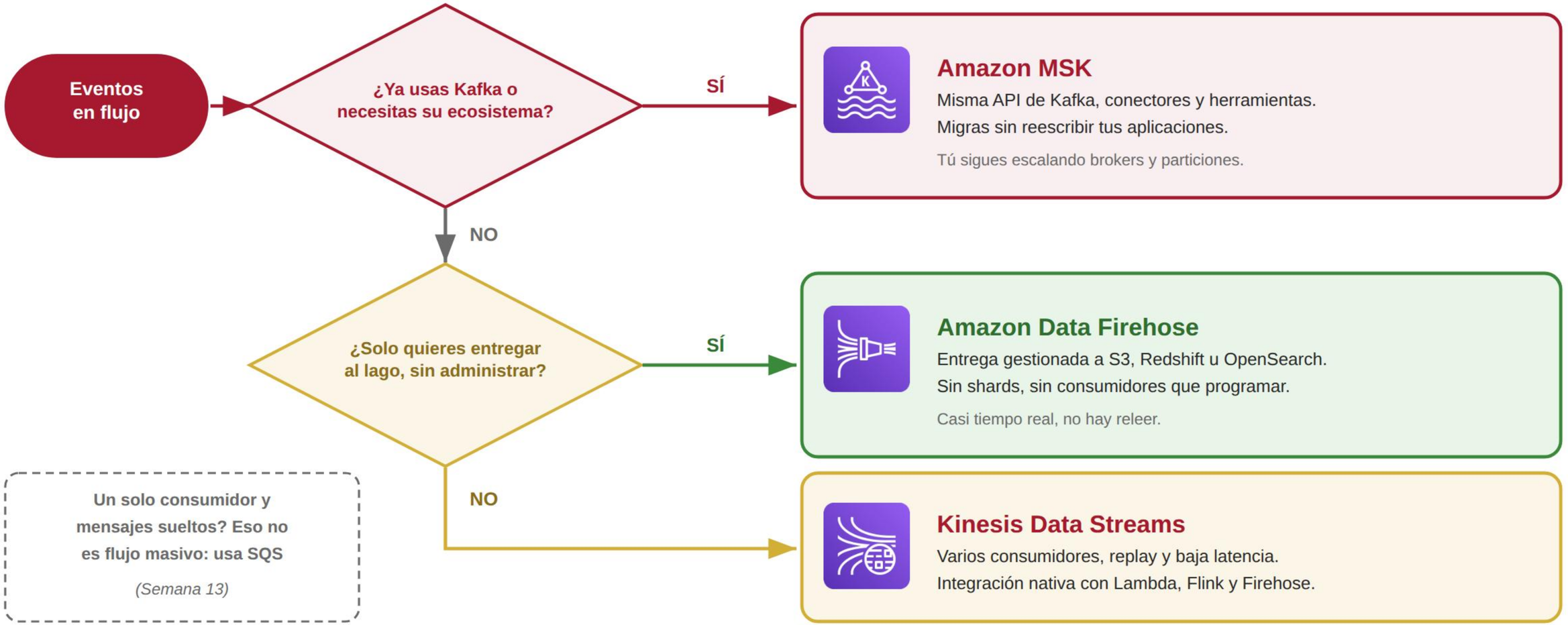


Fuente: adaptado de AWS — Amazon MSK Developer Guide; Apache Kafka

CÓMO ELEGIR

Árbol de decisión: Kinesis, Firehose o MSK

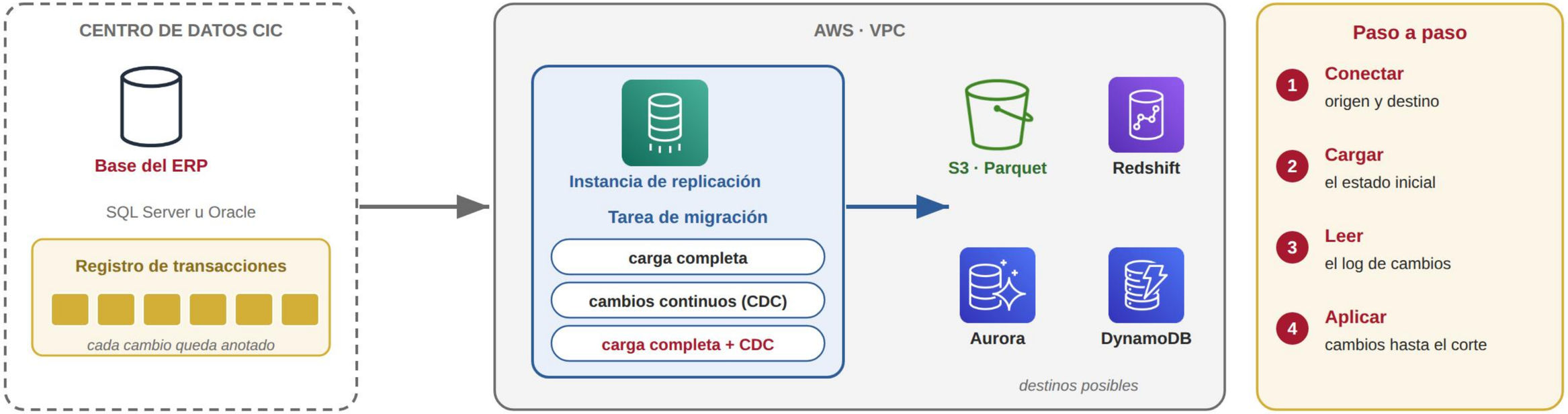
Dos preguntas bastan para escoger el servicio de ingesta en flujo.



Fuente: adaptado de AWS Prescriptive Guidance — streaming data ingestion; criterio propio

DMS: replicar la base del ERP sin detenerla

Primero se copia el estado inicial y luego se aplican los cambios hasta decidir el corte.



Línea de tiempo de la migración: el ERP no se detiene

Carga completa (millones de filas) — *ERP operando normalmente →*

Cambios continuos (CDC): solo lo que se inserta, actualiza o borra

corte a la nube

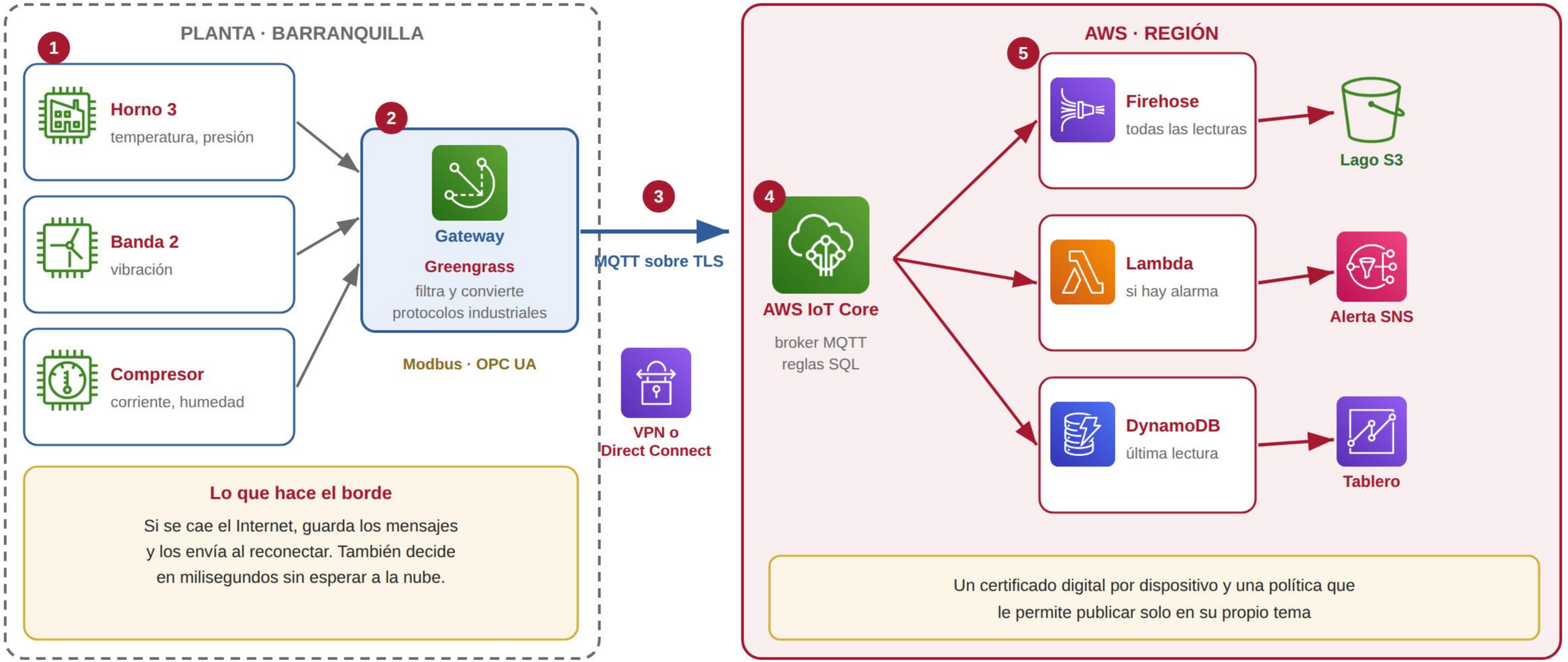
Con DMS Schema Conversion se adapta el esquema cuando el motor de destino es distinto (por ejemplo, Oracle a PostgreSQL).

Fuente: adaptado de AWS Database Migration Service User Guide

CASO APLICADO

Sensores en la planta: del horno a la nube

El gateway de borde traduce, filtra y protege la conexión de la planta con AWS IoT Core.

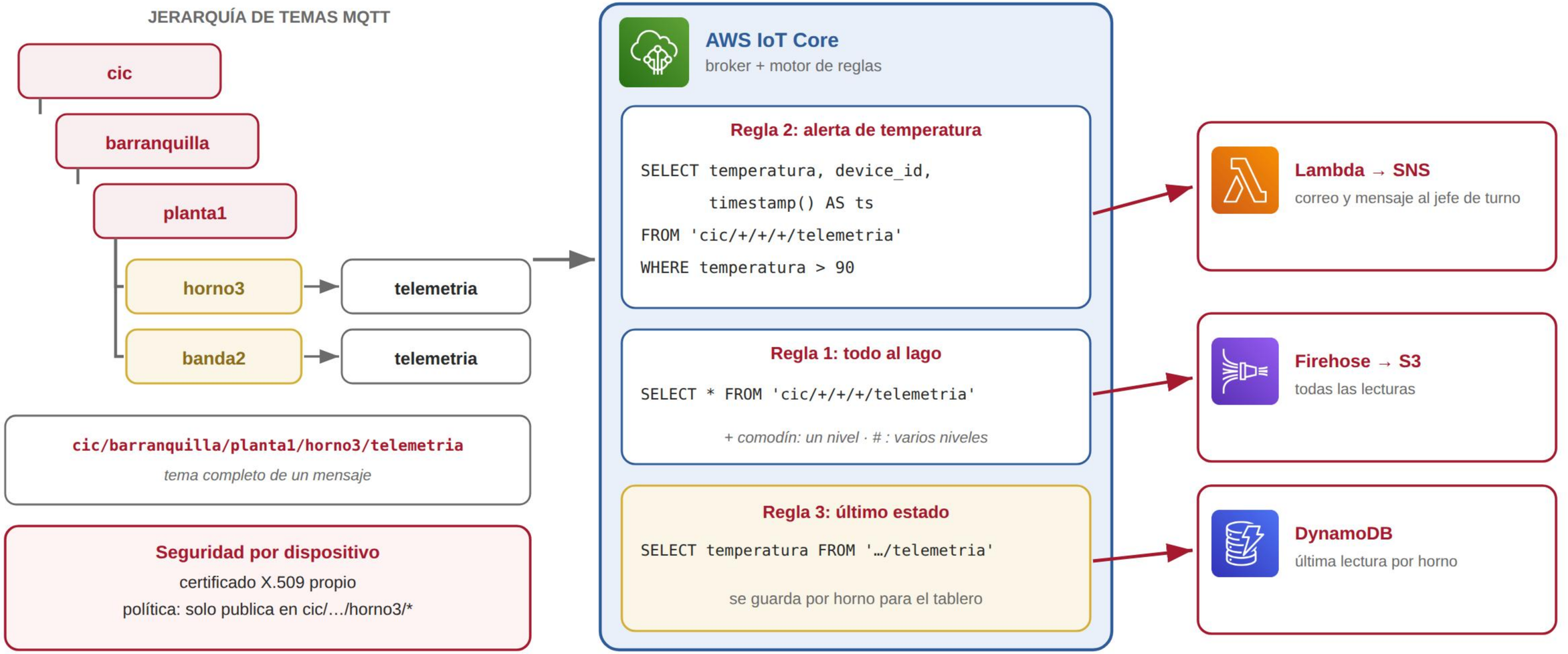


Fuente: adaptado de AWS IoT Core y AWS IoT Greengrass (documentación AWS); caso CIC

CASO APLICADO

IoT Core: temas MQTT y reglas que reparten los datos

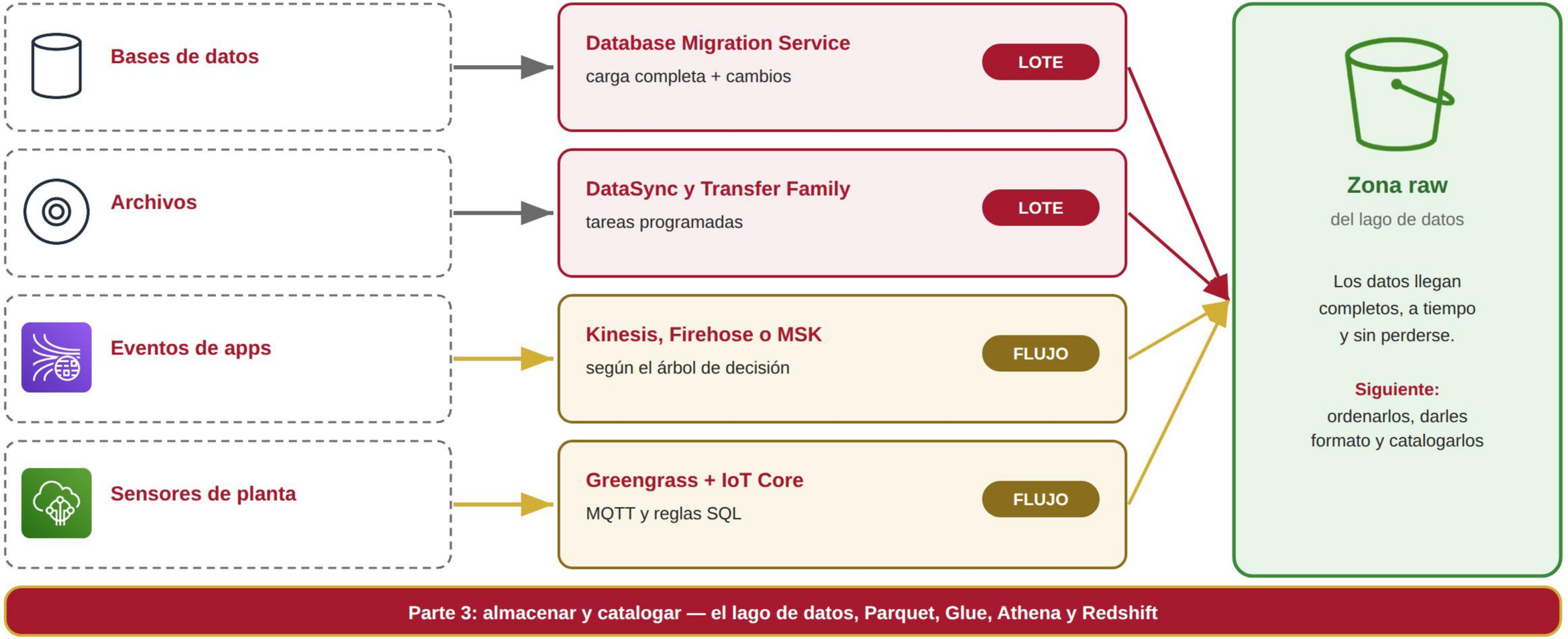
Cada sensor publica en un tema y las reglas SQL deciden a quién le llega cada mensaje.



Fuente: adaptado de AWS IoT Core Developer Guide — MQTT topics y AWS IoT rules

Lo que queda de la Parte 2

Cuatro tipos de origen, dos ritmos y un mismo destino: la zona raw del lago.



Parte 3: almacenar y catalogar — el lago de datos, Parquet, Glue, Athena y Redshift

Fuente: síntesis propia del curso; íconos AWS Architecture Icons

SEMANA 15 · PATRONES DE INGENIERÍA DE DATOS

Almacenar y catalogar: el lago de datos y el almacén

Parte 3 de 4 · Unidad 3

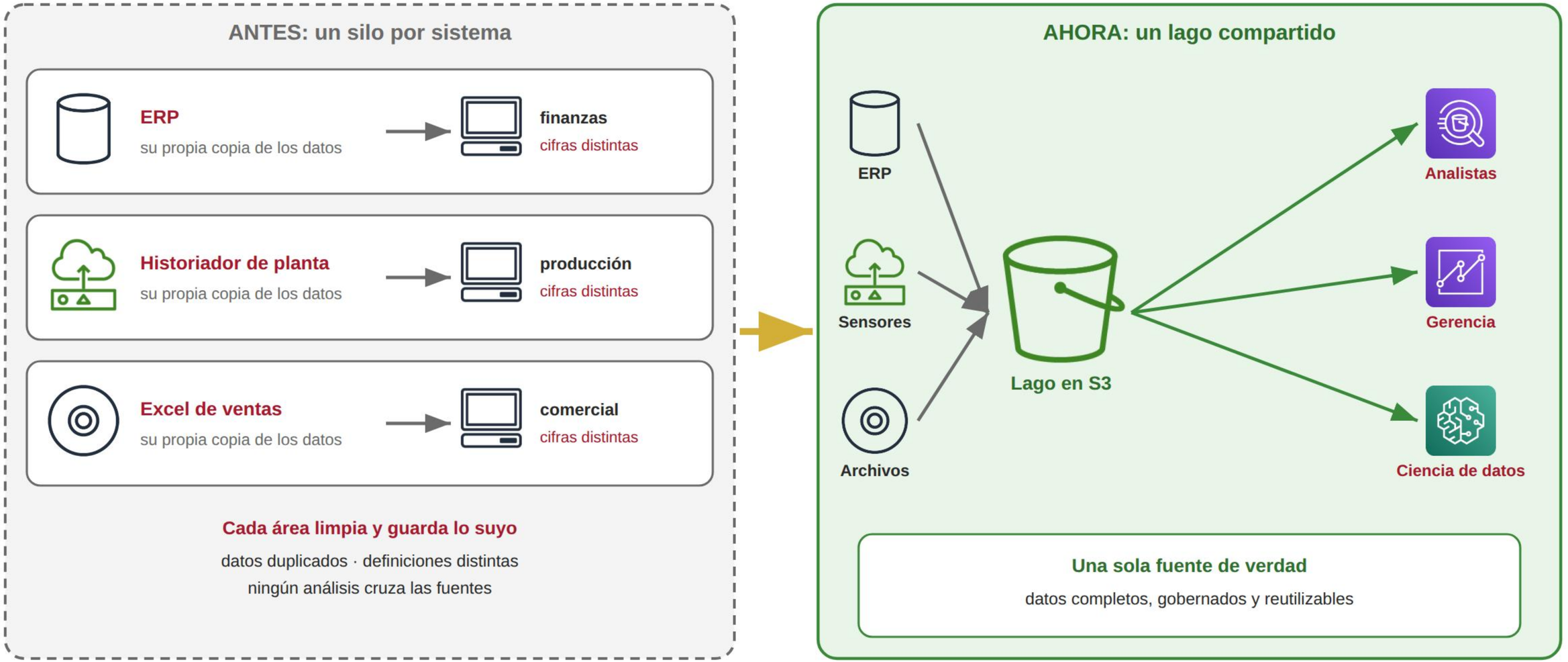
Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 15 · Periodo 2026-2 · Barranquilla, Colombia

Zonas del lago en S3, Parquet, catálogo y ETL con Glue, gobierno con Lake Formation, Athena y Redshift.

ENCUADRE

Del silo al lago: una sola fuente de verdad

Cuando cada área guarda su copia, las cifras no cuadran; el lago concentra y gobierna los datos.



Fuente: adaptado de AWS — What is a data lake?; caso CIC

Amazon S3: la base del lago de datos

Es durable, elástico, acepta cualquier formato y se puede abaratar con reglas de ciclo de vida.

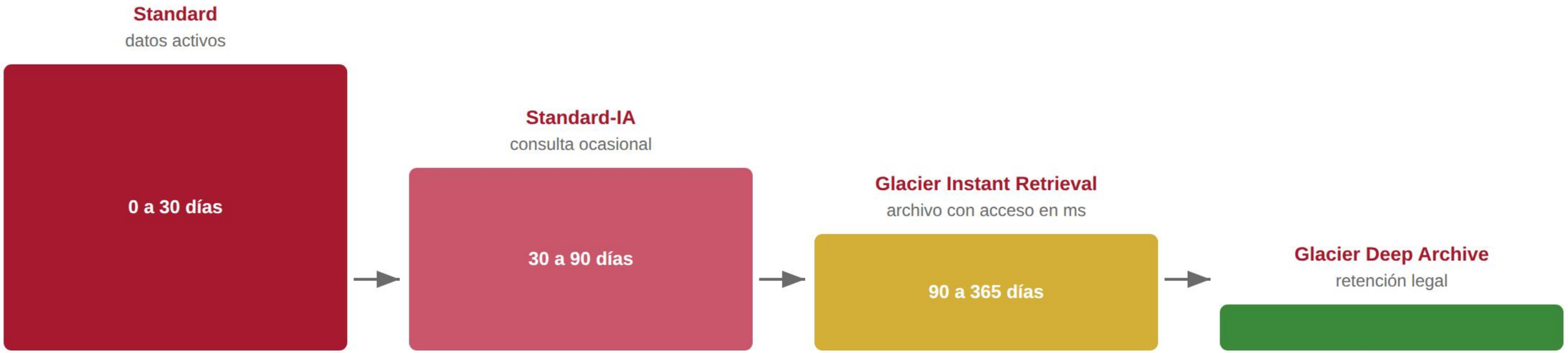
Durable
11 nueves de durabilidad
99,999999999 %

Elástico
crece sin planificar
capacidad ni comprar discos

Cualquier formato
CSV, JSON, Parquet,
imágenes, video

Desacoplado
muchos motores leen
la misma copia

Clases de almacenamiento: el dato envejece y se enfría, la factura baja

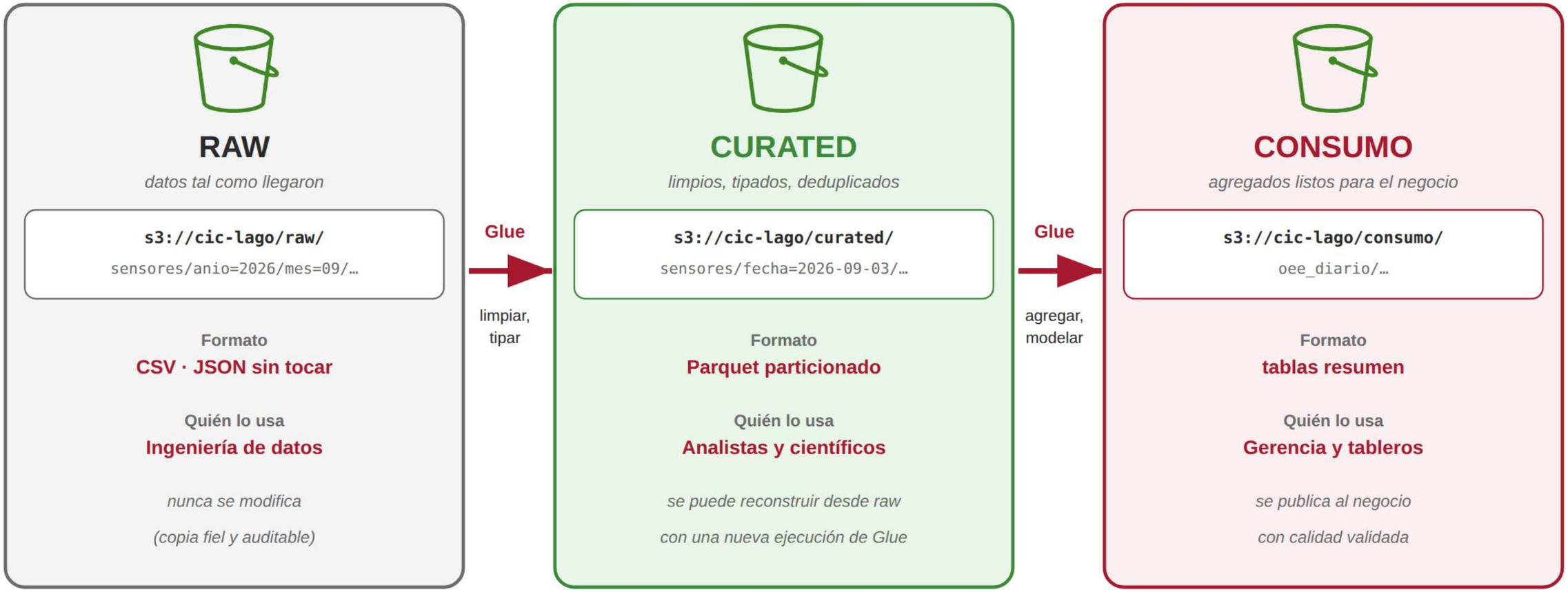


Reglas de ciclo de vida: S3 mueve los objetos solos · plazos ilustrativos, se ajustan por política

Fuente: adaptado de AWS — Amazon S3 storage classes; Building a data lake on AWS

Zonas del lago: raw, curated y consumo

Cada zona sube la calidad: se conserva el original, se limpia y finalmente se publica para el negocio.



Convención de nombres de zonas: también se llaman bronce, plata y oro (arquitectura medallón)

el principio es el mismo: cada zona sube la calidad y baja el volumen

Fuente: adaptado de AWS Prescriptive Guidance — data lake zones; arquitectura medallón

Filas contra columnas: por qué Parquet ahorra

Una consulta analítica casi nunca necesita todas las columnas, y el formato columnar lo aprovecha.

Tabla lógica: 5 lecturas × 6 columnas

`SELECT AVG(temp)` → solo necesita **una columna**

sensor	fecha	temp	presión	humedad	turno
H3-T01	03/09	87,4	1,2	61	A
…	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	…

Formato por filas (CSV, JSON)

cada fila se guarda completa



lee todo

para leer la columna temp hay que recorrer toda la fila: lee el 100 % de los bytes

Formato por columnas (Parquet)

cada columna se guarda junta



lee solo temp

solo se lee el bloque de temp: ≈ 1/6 de los bytes, y comprime mejor porque los valores se parecen

Fuente: adaptado de Apache Parquet — documentación; AWS Prescriptive Guidance — Parquet

Parquet por dentro: bloques con estadísticas

Cada bloque guarda su mínimo y su máximo, así el motor descarta lo que no necesita.

PAR1 · marca de inicio del archivo Parquet

Grupo de filas 1 ~ 128 MB

sensor	fecha	temp	presión	turno
bloque de columna	bloque de columna	bloque de columna	bloque de columna	bloque de columna

Grupo de filas 2 ~ 128 MB

sensor	fecha	temp	presión	turno
bloque de columna	bloque de columna	bloque de columna	bloque de columna	bloque de columna

Pie del archivo (footer): metadatos

esquema · ubicación de cada bloque · mínimo y máximo por bloque

PAR1 · marca de cierre

Lectura inteligente: salto de bloques

Consulta: `WHERE temp > 90`

Grupo de filas 1

`temp: mínimo 71 · máximo 88`

ningún valor supera 90: se SALTA

Grupo de filas 2

`temp: mínimo 74 · máximo 97`

puede haber valores > 90: se LEE



Athena
Redshift Spectrum
EMR

Leen primero el pie del archivo, deciden qué bloques necesitan y descartan el resto sin leerlo.

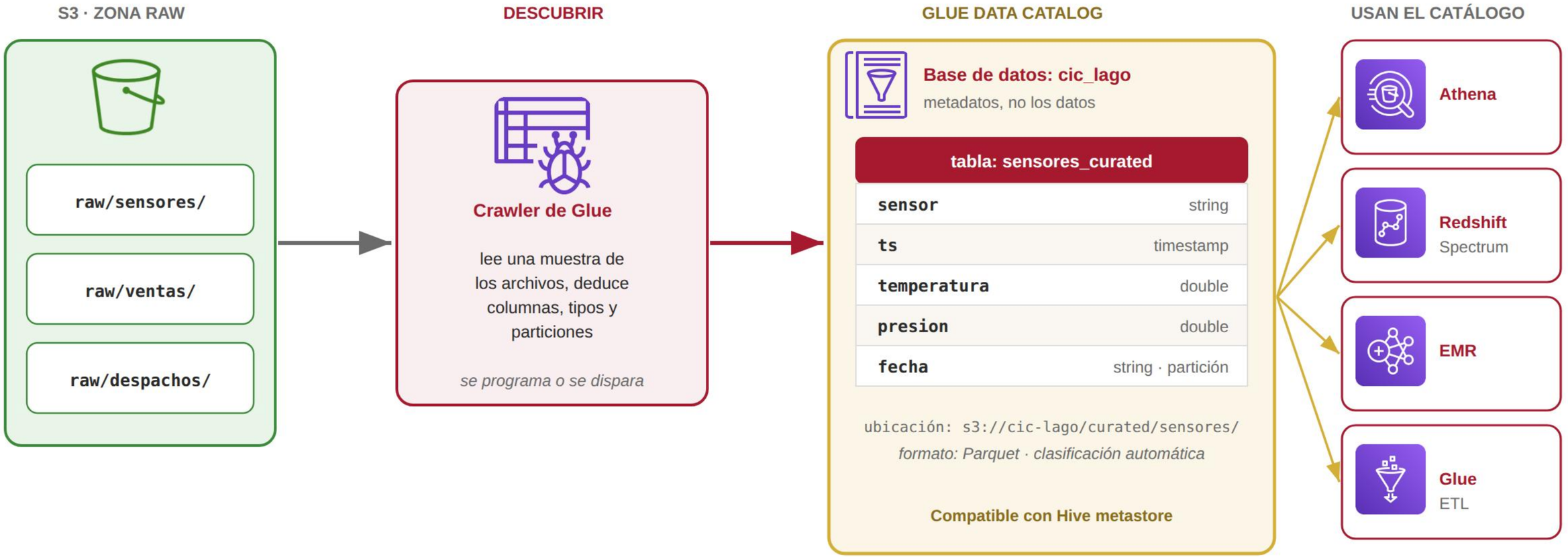
Se llama predicate pushdown (filtro empujado al archivo)

Fuente: adaptado de Apache Parquet — file format specification

CATÁLOGO

AWS Glue: el catálogo que convierte archivos en tablas

Un crawler descubre el esquema de los archivos y lo registra para que todos los motores lo usen.



Un solo catálogo para todos los motores: el esquema se define una vez y todos lo reutilizan

sin catálogo, los archivos son solo bytes; con catálogo, son tablas

Fuente: adaptado de AWS Glue Developer Guide — Data Catalog and crawlers

ETL con AWS Glue: de raw a curated

Un trabajo de Glue limpia, convierte y guarda en Parquet sin que administres un clúster.

raw

CSV · JSON

duplicados, nulos,
unidades mezcladas
y cédulas visibles

```
H3-T01,03/09,,°F
H3-T01,03/09,,°F
```

Trabajo de AWS Glue

Apache Spark sin servidores que administrar

1. Leer los archivos de raw
2. Quitar duplicados y filas inválidas
3. Convertir °F a °C y tipar columnas
4. Enmascarar la cédula (dato personal)
5. Escribir Parquet particionado por fecha

curated

Parquet

limpio y tipado
particionado por fecha

```
fecha=2026-09-03/
fecha=2026-09-04/
fecha=2026-09-05/
```

```
df = spark.read.csv("s3://cic-lago/raw/sensores/", header=True)
df = df.dropDuplicates(["sensor", "ts"]).filter("temp IS NOT NULL")
df = df.withColumn("temp", (df.temp - 32) * 5 / 9) # solo filas en °F
(df.write.partitionBy("fecha")
   .parquet("s3://cic-lago/curated/sensores/"))
```

Esquema simplificado en PySpark: la lógica es el ejemplo, no un script de producción

Cómo se dispara

- Programado: cada noche
- Evento: llegó un archivo nuevo
- Flujo de trabajo encadenado

Fuente: adaptado de AWS Glue Developer Guide — ETL jobs; ejemplo propio del caso CIC

Lake Formation: quién ve qué columna

Los permisos se definen una vez, por columna y por fila, y todos los motores los respetan.

Tabla clientes del lago: ¿qué columnas ve cada rol?

	nombre	cédula	ciudad	ventas
Analista de ventas — solo ciudad y ventas	**sin acceso** bloqueada	**sin acceso** bloqueada	**visible** permitida	**visible** permitida
Científico de datos — cédula con hash irreversible	**sin acceso** bloqueada	**enmascarada** hash	**visible** permitida	**visible** permitida
Auditor interno — todo, con registro de acceso	**visible** permitida	**visible** permitida	**visible** permitida	**visible** permitida

AWS Lake Formation

Permisos centrales por base, tabla, columna y fila, con etiquetas (LF-tags)

Se otorga una vez y lo respetan Athena, Redshift Spectrum, EMR y Glue

Cómo se aplica la regla



la consulta llega a Athena; Lake Formation entrega credenciales solo para lo permitido

Fuente: adaptado de AWS Lake Formation Developer Guide — fine-grained access control

Athena: SQL directo sobre los archivos de S3

No hay servidor que administrar: se escribe SQL, Athena lee S3 y cobra por lo escaneado.

Analista

Editor de consultas de Athena

```
SELECT planta,
       AVG(temperatura) AS temp_media
FROM cic_lago.sensores_curated
WHERE fecha = '2026-09-03'
GROUP BY planta;
```

Amazon Athena

serverless
motor SQL distribuido
(Trino)

1

Glue Data Catalog

¿qué columnas y dónde está
la tabla?

Amazon S3 · zona curated

solo lee los archivos Parquet de fecha=2026-09-03

… … … **03/09** … … …

Resultado y costo

planta	temp_media
Planta 1	84,3
Planta 2	79,8

Se paga por los datos escaneados (≈ 5 USD por TB, ilustrativo)

Sin servidores · sin clústeres · SQL estándar sobre archivos en S3

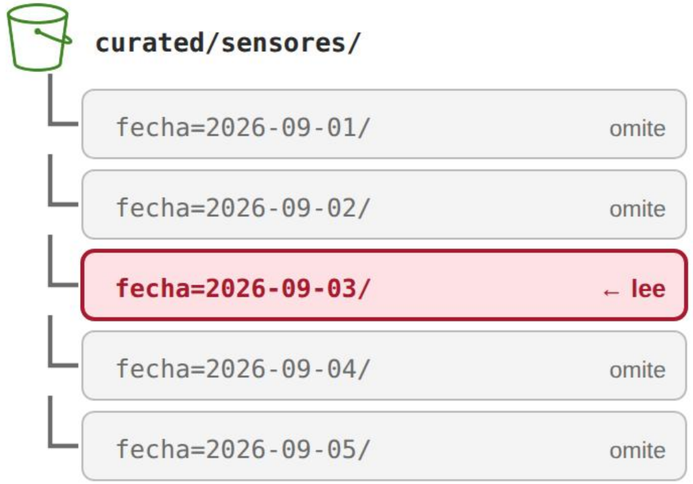
ideal para explorar datos de forma ocasional o ad hoc

Fuente: adaptado de Amazon Athena User Guide; precio ilustrativo, varía por región

Optimizar consultas: formato, partición y tamaño

Los mismos datos pueden costar mil veces menos de consultar si se organizan bien.

Particiones: se lee solo la carpeta necesaria



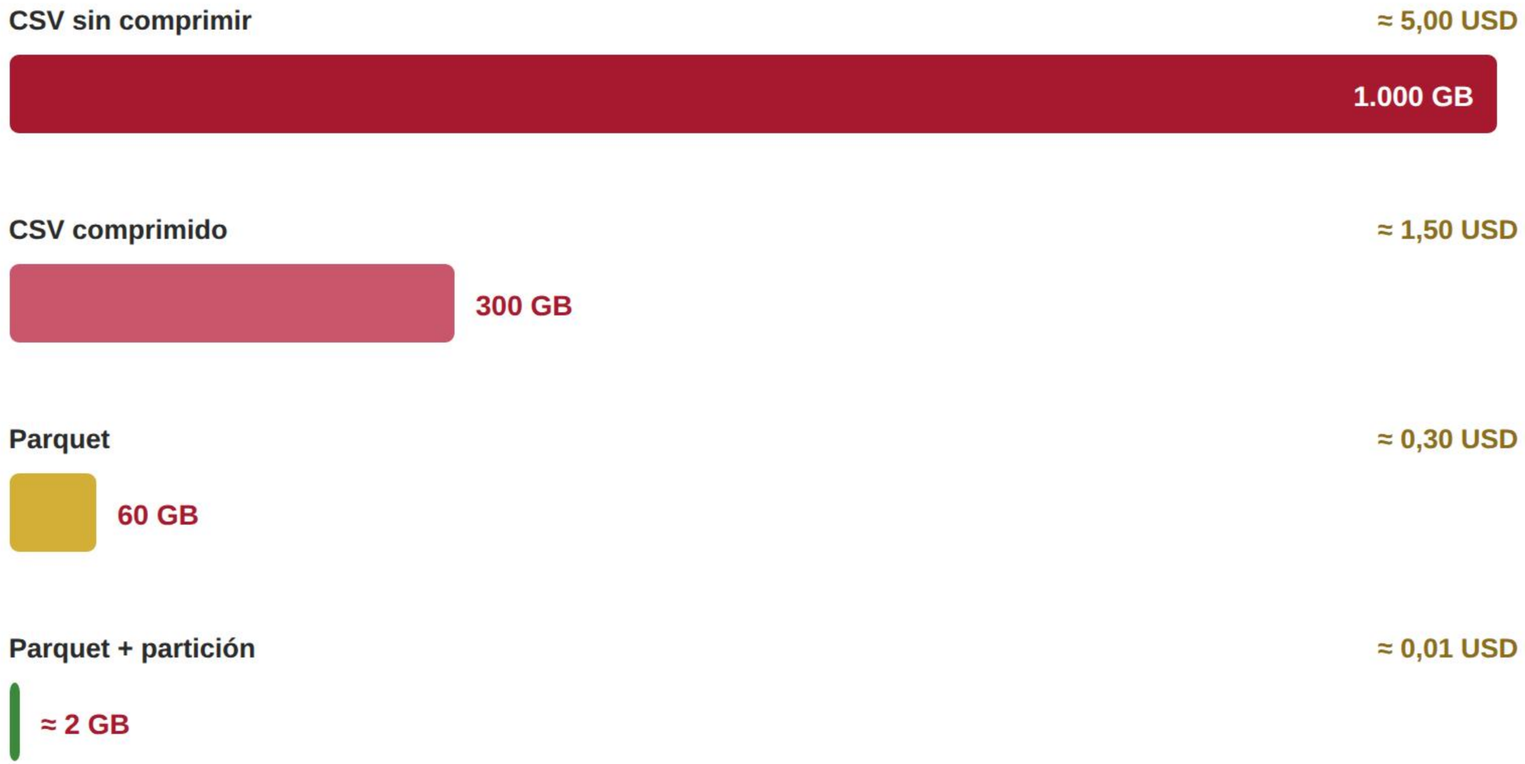

```
WHERE fecha = '2026-09-03'
```

1 carpeta de 5 (o de 365)

Cinco hábitos que bajan la factura

1 Parquet en vez de CSV
2 comprimir (Snappy o ZSTD)
3 particionar por la columna de filtro
4 archivos de 128 MB o más, no miles de pequeños

Datos escaneados por la misma consulta (ilustrativo, 1 TB de CSV)

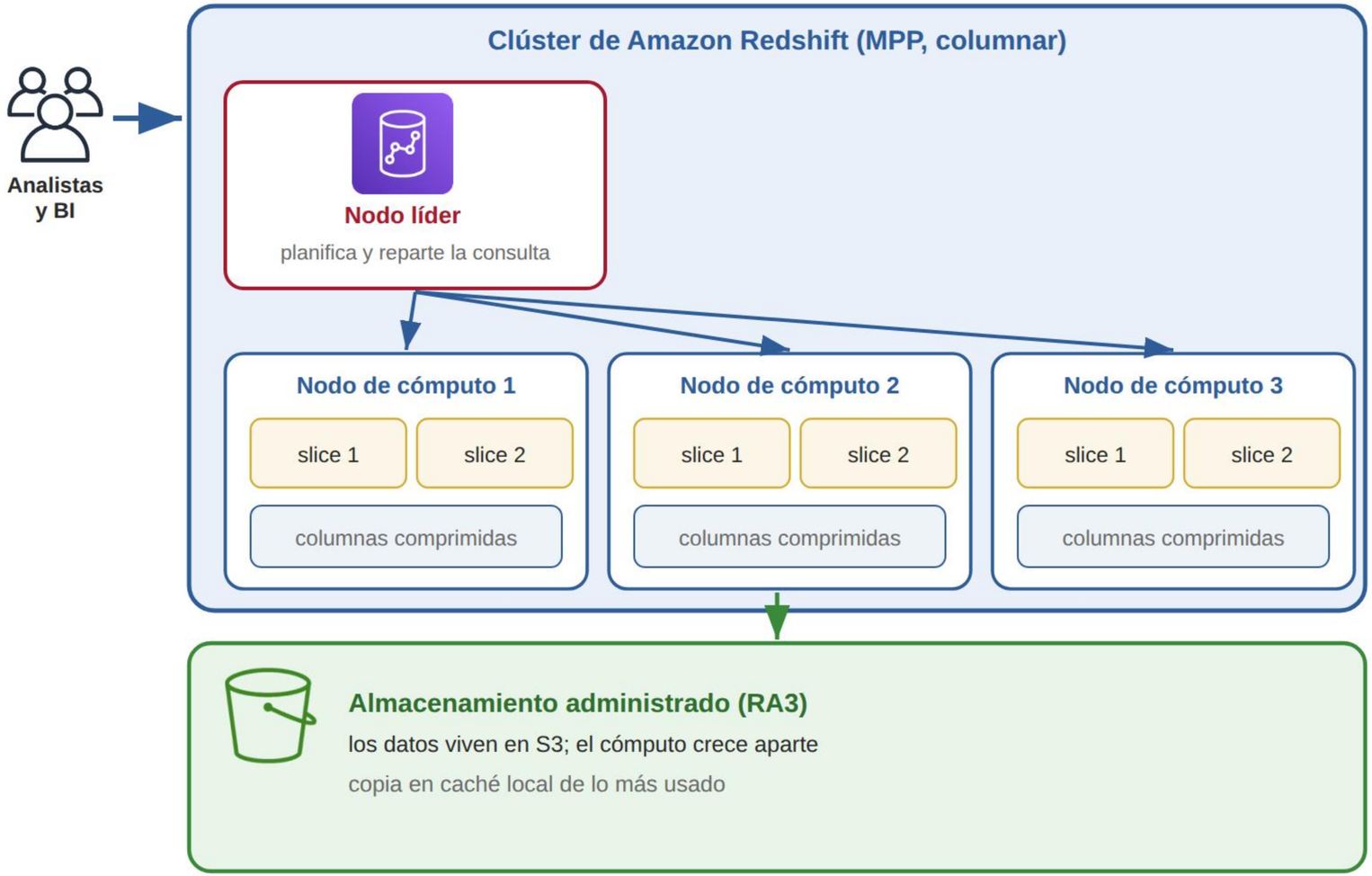


precio por TB escaneado ilustrativo: 5 USD · la reducción real depende de columnas y filtros

Fuente: adaptado de AWS Prescriptive Guidance — best practices for Athena; cifras ilustrativas

Amazon Redshift: el almacén de datos de AWS

Un nodo líder reparte la consulta entre nodos de cómputo que leen columnas comprimidas en paralelo.



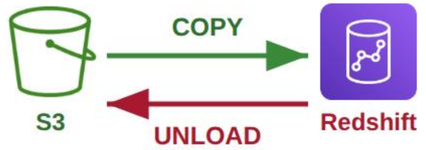
Opciones de despliegue

Aprovisionado
eliges tipo y número de nodos

Serverless
escala solo, pagas por uso (RPU)

Concurrency Scaling
capacidad extra en picos de consultas

Conexión con el lago



Redshift Spectrum
consulta tablas del lago
sin copiarlas al almacén

Fuente: adaptado de Amazon Redshift Database Developer Guide

COMPARAR

Lago contra almacén: esquema al leer o al escribir

El lago es flexible y barato; el almacén es veloz y predecible. Se complementan.

LAGO DE DATOS

esquema al leer (schema-on-read)

CSV · JSON · Imágenes → **se guarda tal cual**

sin definir columnas

El esquema se aplica al consultar

el mismo archivo puede leerse con esquemas distintos
según la pregunta de cada equipo

Flexibilidad	●●●●●
Costo de almacenar	●●●●●
Velocidad en consultas	●●●○○
Datos no estructurados	●●●●●

ALMACÉN DE DATOS

esquema al escribir (schema-on-write)

Tablas → ETL → Modelo → **tablas con esquema fijo**

claves, tipos y relaciones

El esquema se valida antes de guardar

los datos ya llegan limpios, modelados y optimizados
para consultas repetitivas de muchos usuarios

Flexibilidad	●●○○○
Costo de almacenar	●●●○○
Velocidad en consultas	●●●●●
Datos no estructurados	●○○○○

No es una pelea: son dos herramientas para dos necesidades

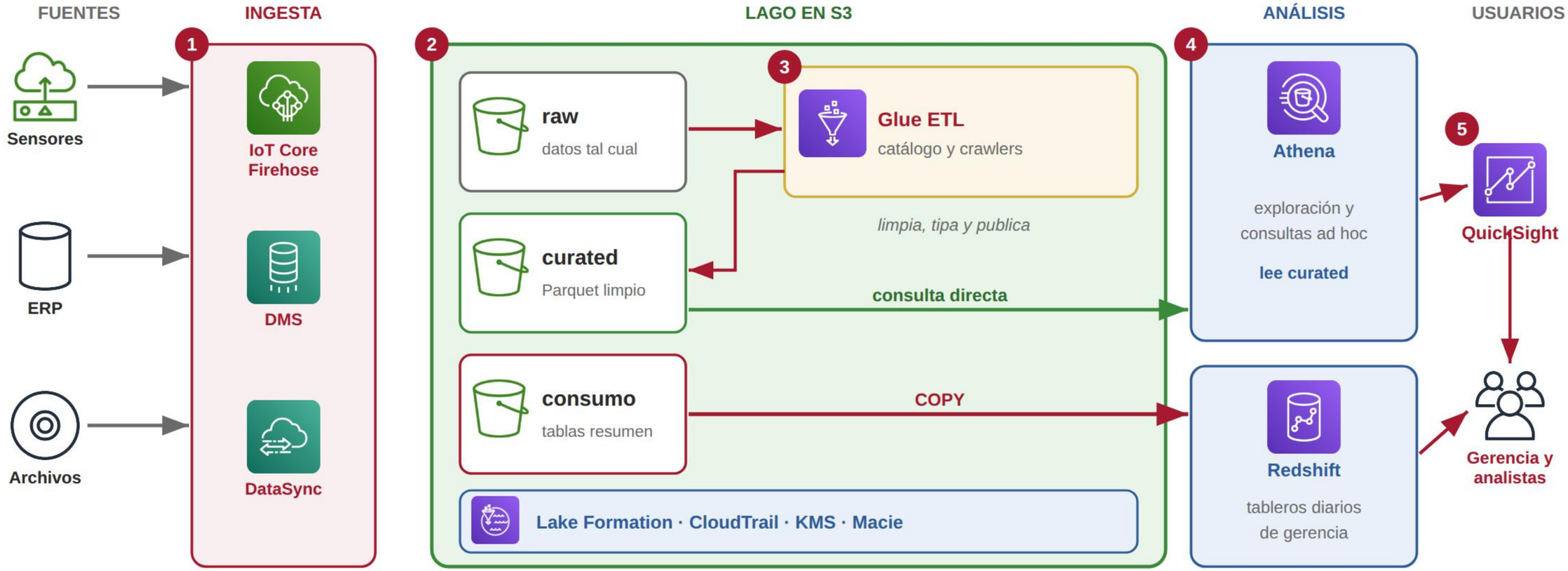
el lago explora y guarda todo; el almacén sirve rápido lo ya validado · la tendencia actual los combina (lakehouse)

Fuente: adaptado de AWS — Data lake vs data warehouse; criterio propio del curso

CASO APLICADO

Arquitectura combinada: lago y almacén en la CIC

El lago guarda todo y explora con Athena; el almacén sirve rápido los tableros que gerencia consulta cada día.



1 ingresa 2 aterriza en raw 3 Glue limpia, cataloga y publica 4 Athena explora y Redshift sirve 5 se visualiza

Spectrum permite que Redshift lea el lago sin copiar · Lake Formation gobierna ambos motores

Fuente: adaptado de AWS Architecture Center — Modern data architecture; caso CIC

Lo que queda de la Parte 3

Con el lago, el catálogo y los motores de consulta, los datos ya están listos para analizarse.

1



Lago en S3

tres zonas: raw, curated y consumo

2



Parquet y catálogo

menos bytes leídos y tablas reutilizables

3



Gobierno

permisos por columna y por fila

4



Athena y Redshift

explorar o servir, según la necesidad



Ingestar

Parte 2

→



Almacenar

Parte 3

→



Procesar

Parte 3

→



Analizar

Parte 3

→



Visualizar

Parte 4

Lo que sigue: la Parte 4 cierra el pipeline

Visualizar con QuickSight, analítica en tiempo real, arquitectura moderna de referencia y el caso completo

Fuente: síntesis propia del curso; íconos AWS Architecture Icons

SEMANA 15 · PATRONES DE INGENIERÍA DE DATOS

Analizar y visualizar: la arquitectura moderna de datos

Parte 4 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 15 · Periodo 2026-2 · Barranquilla, Colombia

QuickSight, analítica en tiempo real, arquitectura de referencia y el caso completo de la Corporación Industrial del Caribe.

Del lago a la decisión: cada pregunta, su herramienta

Cuatro personas hacen cuatro preguntas distintas sobre los mismos datos del lago.



Fuente: adaptado de AWS — Modern Data Architecture; casos CIC

Motores de análisis: de sin servidores a control total

Se empieza por lo más administrado y se baja a más control solo cuando la carga lo exige.

MENOS ADMINISTRACIÓN → MÁS CONTROL



Athena

SQL ad hoc sobre S3

sin servidores
pago por TB escaneado

explorar



AWS Glue

ETL con Spark

sin clúster
pago por tiempo de ejecución

transformar



Redshift Serverless

SQL de BI y concurrencia

escala solo
pago por uso

tableros de BI



EMR Serverless

Spark y Hive a medida

eliges versión y librerías

Spark a medida



EMR en EC2

control total del clúster

Spark, Flink, HBase
tú ajustas nodos

ajuste fino

¿Cómo se escoge?

empieza por el extremo sin servidores; baja a más control solo si la carga lo exige

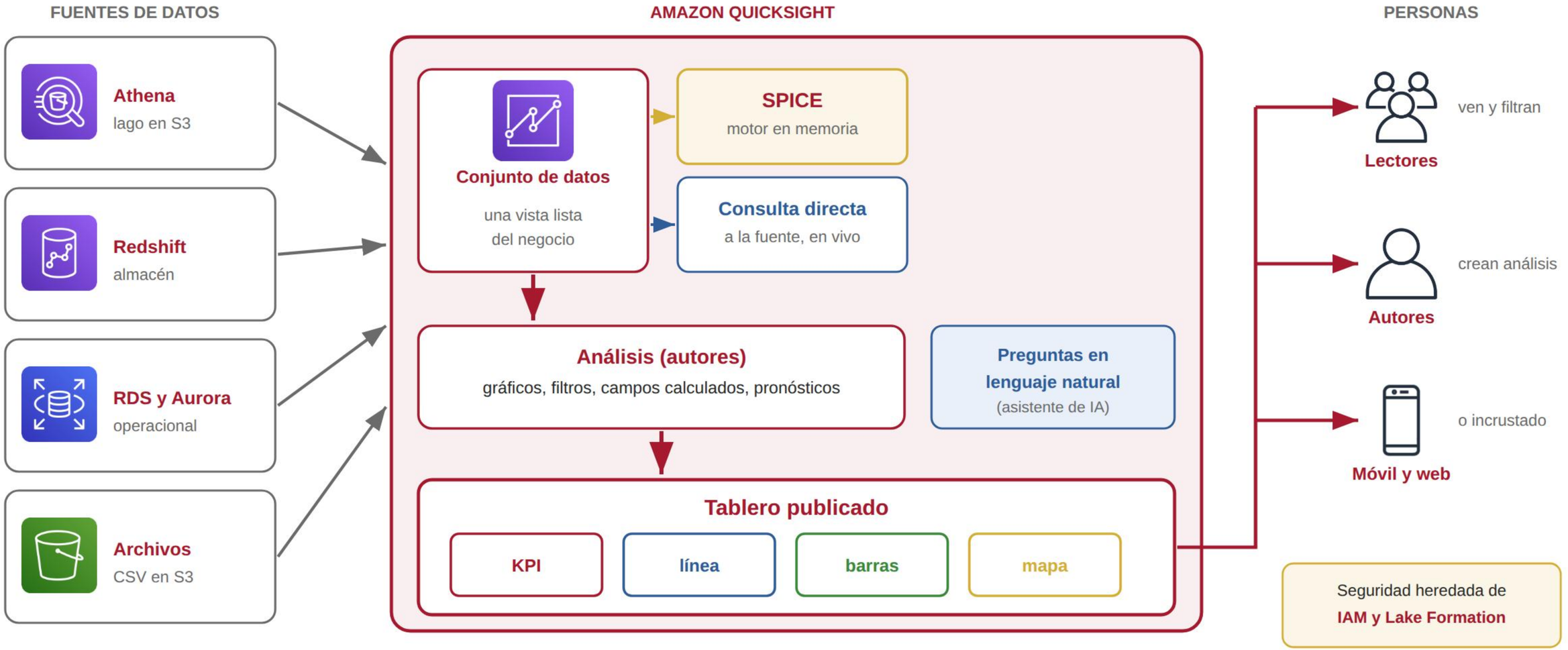
cada escalón hacia la derecha agrega flexibilidad, pero también trabajo de operación

Fuente: adaptado de AWS — Choosing an analytics service; Data Analytics Lens

VISUALIZAR

Amazon QuickSight: del conjunto de datos al tablero

Se conecta a las fuentes, modela un conjunto de datos y publica tableros para lectores.



Fuente: adaptado de Amazon QuickSight User Guide

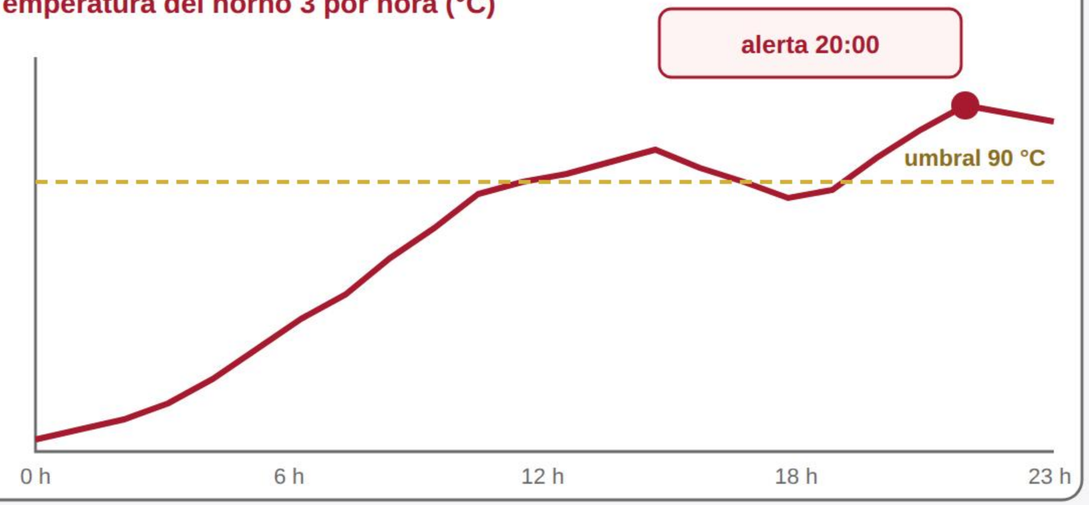
Un tablero para la planta: tres lecturas en segundos

Indicadores arriba, tendencia con umbral y causas ordenadas: el tablero responde antes de que se pregunte.

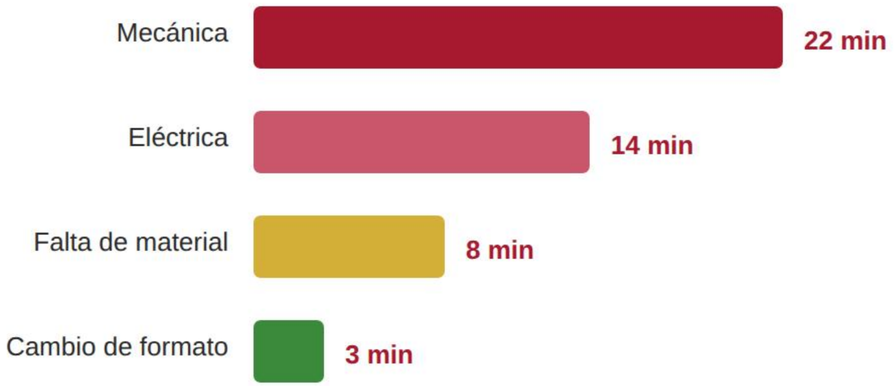
Corporación Industrial del Caribe · Planta Barranquilla · Tablero de producción — turno A · hoy

OEE de la planta	Paradas hoy	Merma	Energía del día
78 % ▲ 2 puntos	3 — 47 minutos	2,1 % ▼ 0,4 puntos	12,4 MWh ▲ 3 %

Temperatura del horno 3 por hora (°C)



Minutos de parada por causa



1. **KPI de un vistazo**
2. **tendencia con umbral visible**
3. **causas ordenadas de mayor a menor**

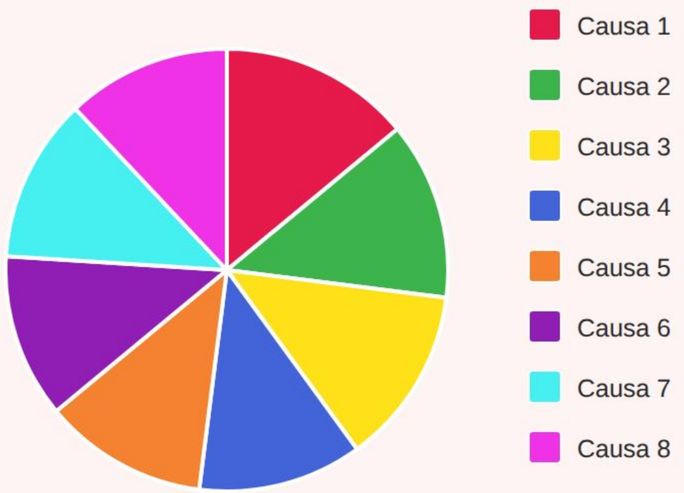
cifras ilustrativas

Fuente: maqueta propia con cifras ilustrativas; criterios de diseño de QuickSight

Buenas prácticas: que el gráfico se entienda solo

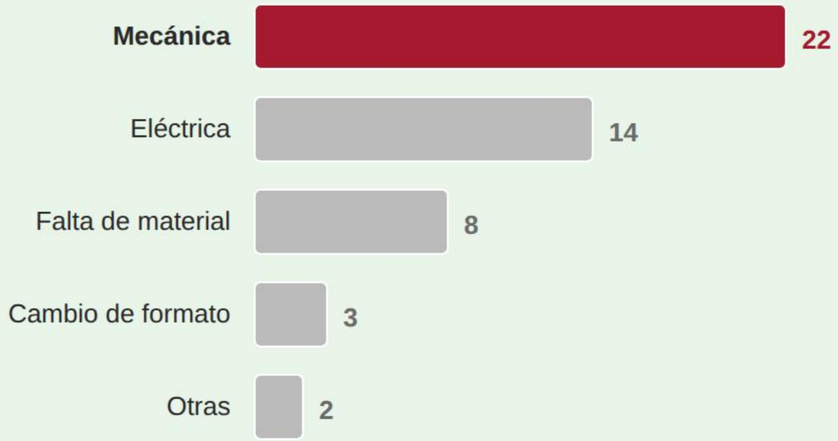
Menos color, más orden: un gráfico claro se lee en cinco segundos.

Difícil de leer



8 colores, sin orden, sin valores: ¿cuál es la mayor?

Claro en cinco segundos



ordenado, con valores y un solo color de énfasis

1 Una pregunta por gráfico

el título es la conclusión

2 Ordenar de mayor a menor

salvo series de tiempo

3 Color con propósito

resaltar lo importante, no decorar

4 Etiquetas directas

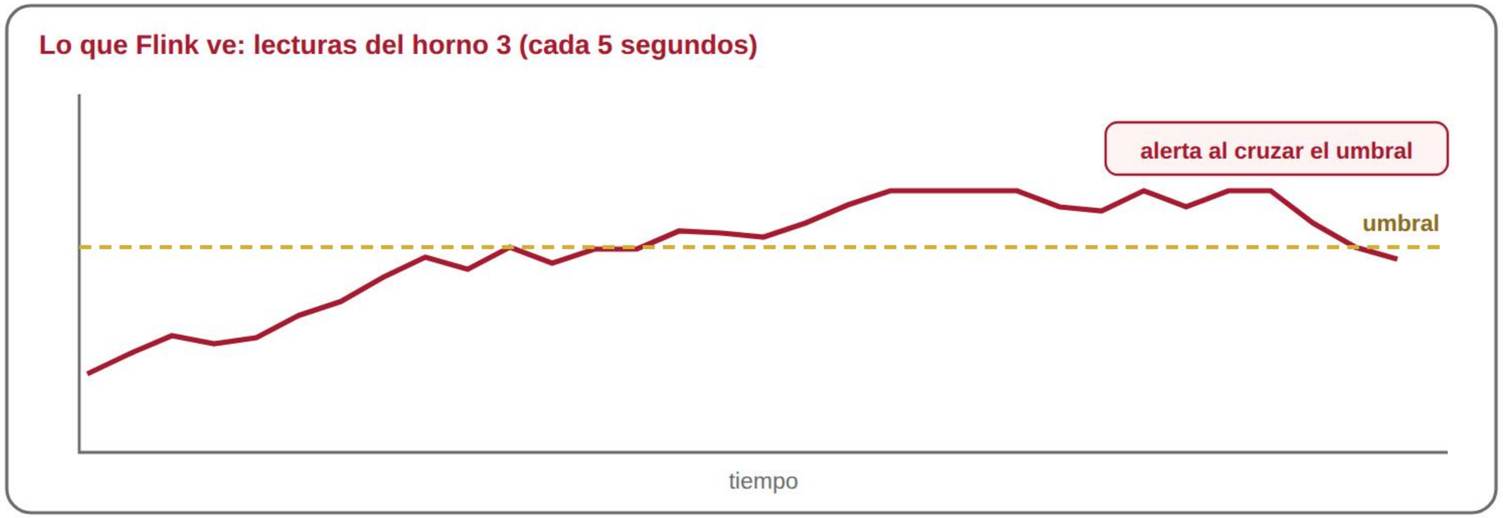
sin obligar a mirar la leyenda

Fuente: adaptado de principios de visualización de datos (Few, Tufte) y guías de QuickSight

Analítica en tiempo real: del sensor a la alerta en segundos

Flink calcula sobre el flujo y dispara acciones sin esperar a que lleguen los lotes.





Cuándo vale la pena

Alertas de seguridad y calidad

Detección de fallas y fraude

Tableros operativos en vivo

Mantenimiento por condición

costo: servicio siempre encendido

Fuente: adaptado de AWS — Real-time analytics with Managed Service for Apache Flink

Ventanas de tiempo: cómo agrupar un flujo infinito

Un flujo no termina; para calcular algo hay que cortarlo en ventanas.

VENTANA FIJA (tumbling)

promedio por minuto

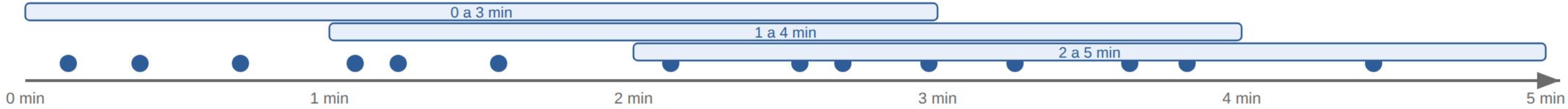
cada evento cae en una sola ventana



VENTANA DESLIZANTE (sliding / hopping)

promedio de los últimos 3 min, cada minuto

las ventanas se traslapan y se mueven



VENTANA DE SESIÓN

actividad de un camión hasta una pausa de 10 min

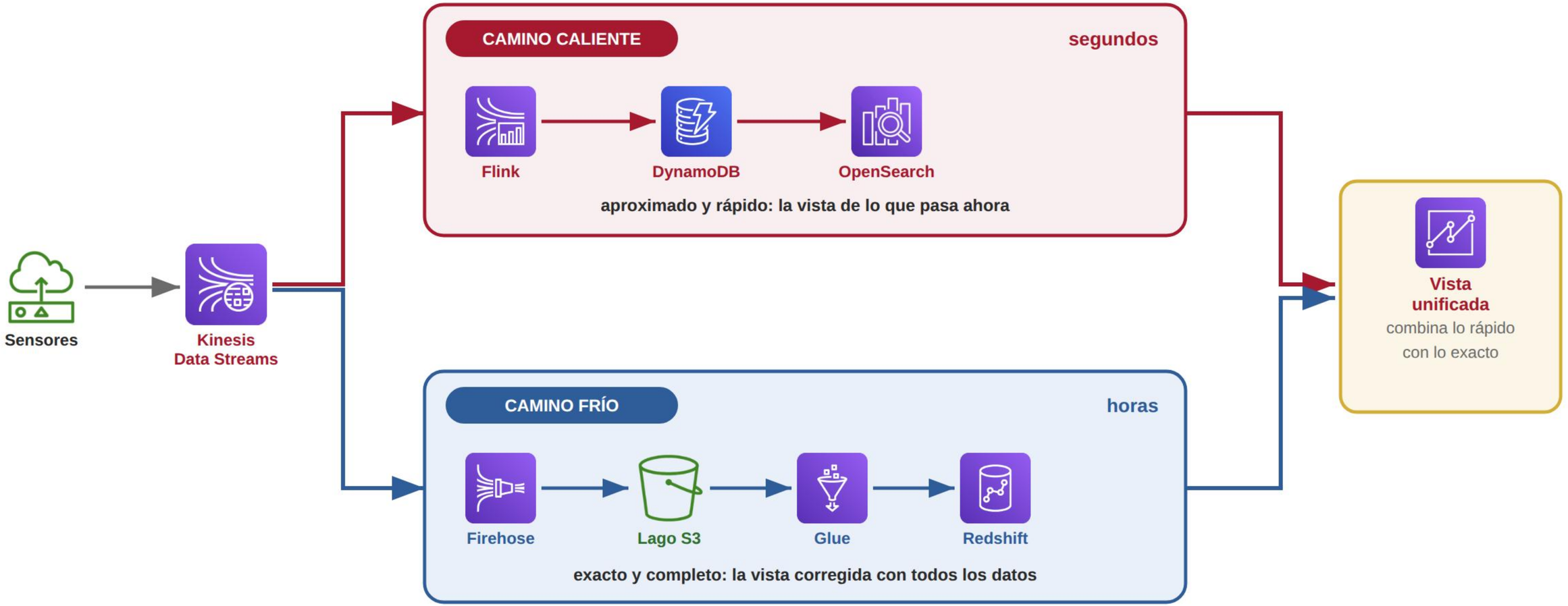
se cierra tras un silencio



Fuente: adaptado de Apache Flink — Windows; AWS Managed Service for Apache Flink

Camino caliente y camino frío sobre el mismo flujo

Un camino responde en segundos con lo aproximado; el otro corrige en horas con lo exacto.

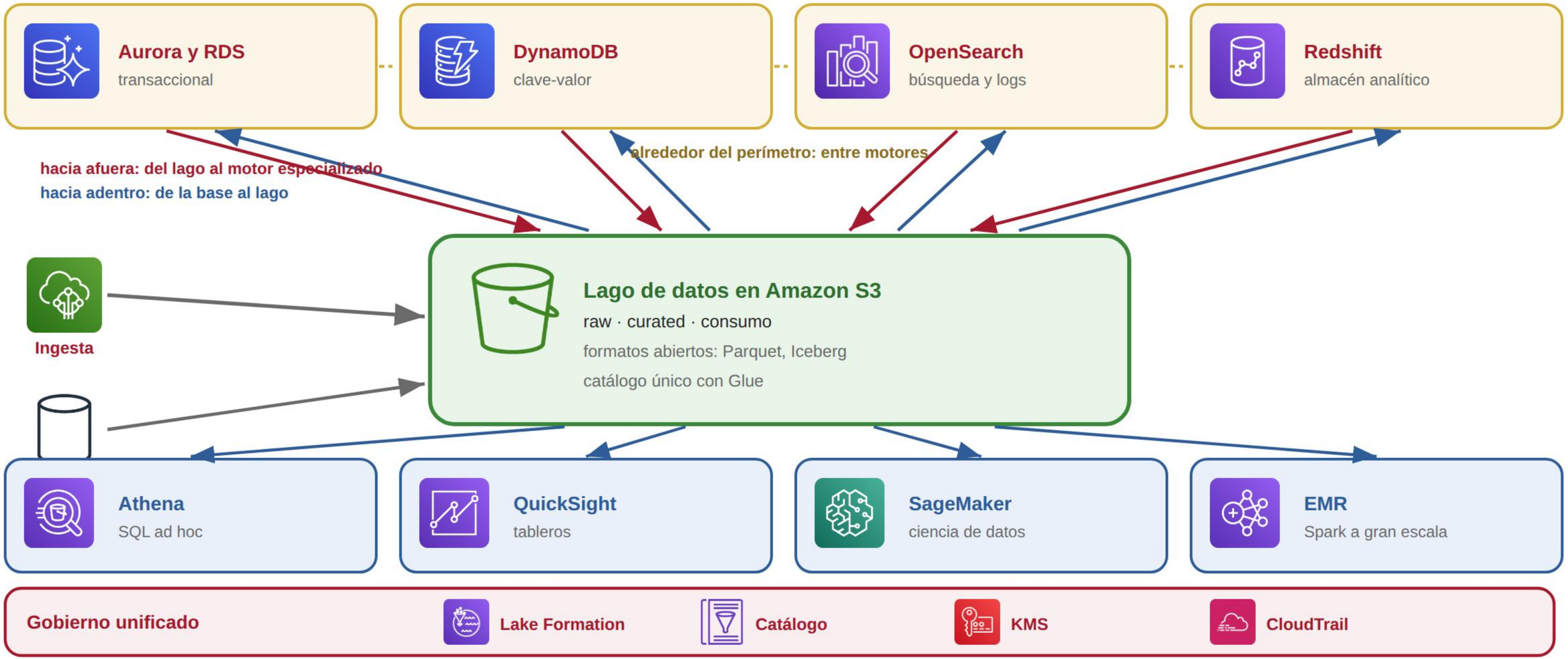


También existe la variante Kappa: todo en flujo y, si hace falta, se vuelve a procesar releyendo el stream

Fuente: adaptado de AWS Big Data Blog — Lambda architecture on AWS; Kappa architecture

Arquitectura moderna de datos: el lago en el centro

Los datos se mueven hacia afuera, hacia adentro y alrededor del perímetro, siempre con un gobierno común.



Fuente: adaptado de AWS — Modern Data Architecture (inside-out, outside-in, around the perimeter)

Bases de propósito específico: cada carga, su motor

No se usa un martillo para todo: cada tipo de dato y consulta tiene un motor que lo resuelve mejor.

Aurora y RDS

Pedidos y facturación

¿Por qué?
transacciones consistentes con SQL

DynamoDB

Estado actual de cada equipo

¿Por qué?
lectura en milisegundos a cualquier escala

Timestream for InfluxDB

Series de tiempo de sensores

¿Por qué?
consultas por intervalos y tendencias

OpenSearch Service

Búsqueda en registros de fallas

¿Por qué?
texto libre, logs y paneles

Redshift

Tableros históricos

¿Por qué?
SQL analítico sobre años de datos

Neptune

Cadena de suministro

¿Por qué?
relaciones entre proveedores y piezas (grafo)

Y todas se conectan con el lago

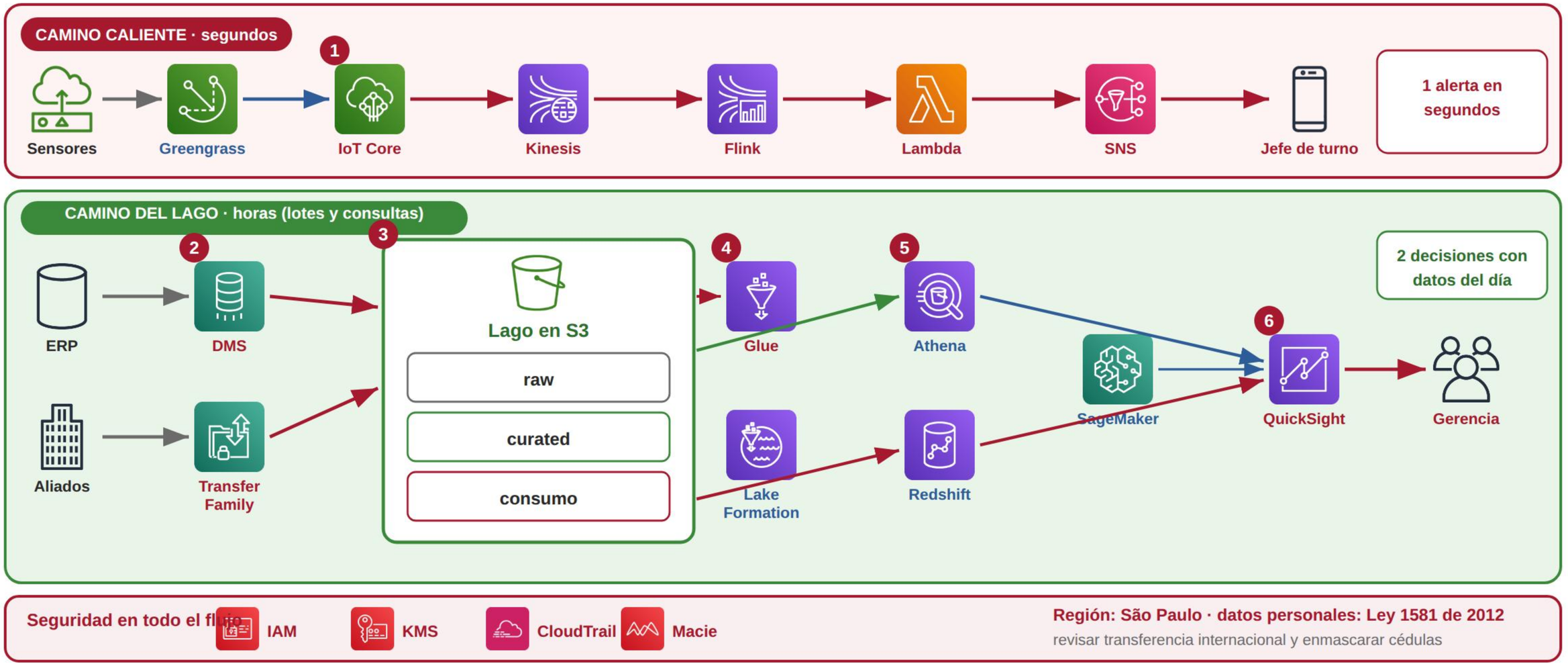
cada base entrega su historia al lago (hacia adentro) y recibe del lago lo que necesita (hacia afuera)

así el lago conserva la historia completa y cada motor sirve su propia carga sin estorbar a los demás

Fuente: adaptado de AWS — Purpose-built databases; caso CIC

El caso completo: la Corporación Industrial del Caribe

Un camino caliente alerta en segundos y el camino del lago decide con datos completos del día.



Fuente: arquitectura integradora basada en AWS Modern Data Architecture; caso CIC

Costo del pipeline: dónde está y cómo bajarlo

Cada etapa tiene palancas propias; medir por etapa permite ahorrar sin perder capacidad.

¿Dónde se va el dinero de un pipeline? (reparto ilustrativo)



Ingesta

Palancas de ahorro

Buffer más grande en Firehose

menos entregas pequeñas

Kinesis bajo demanda si hay picos

15 % del gasto

Almacenamiento

Palancas de ahorro

Reglas de ciclo de vida

Parquet comprimido

borrar copias temporales

20 % del gasto

Procesamiento

Palancas de ahorro

Glue con workers justos

EMR con instancias Spot

trabajos solo cuando hay datos

30 % del gasto

Consulta

Palancas de ahorro

Particiones y Parquet

límite de bytes por grupo

de trabajo en Athena

20 % del gasto

Visualización

Palancas de ahorro

SPICE en lugar de consulta

directa siempre

licencias según el uso real

15 % del gasto

Pilares Optimización de costos y Eficiencia del rendimiento (Well-Architected)

etiquetas por proyecto para ver quién gasta · AWS Budgets para alertas · revisar el pipeline cada trimestre

Fuente: adaptado de AWS Well-Architected — Cost Optimization; reparto ilustrativo propio

EJERCICIO DE AULA

Reto: diseña el pipeline de la banda transportadora 2

Cinco etapas, cinco decisiones: tu diseño debe justificar el ritmo, el costo y el gobierno.

Reto de diseño

Banda transportadora 2

Tiene acelerómetros que envían 1.000 lecturas por segundo cada uno.

a **Alerta si la vibración sube**

b **Reporte semanal por turno**

c **Conservar 3 años por auditoría**

Trabajo en grupos · 10 minutos

dibuja y justifica cada decisión

Ingestar



?

¿Lote o flujo?
¿Streams, Firehose o MSK? ¿Borde?

Almacenar



?

¿Qué zonas?
¿Qué formato?
¿Ciclo de vida?

Procesar



?

¿Glue o Flink?
¿Qué limpiar?
¿Qué agregar?

Analizar



?

¿Athena o Redshift?
¿Con qué ritmo?

Visualizar



?

¿Alerta o tablero?
¿Quién lo ve?
¿Qué permisos?

Criterios con los que se evalúa tu diseño

Ritmo correcto

la latencia que pide la alerta

Costo razonable

sin pagar tiempo real donde no hace falta

Gobierno

quién ve qué, cifrado y audito ría

Fuente: ejercicio propio del curso; íconos AWS Architecture Icons

CIERRE DE LA SEMANA

Semana 15 en cuatro ideas

Los datos, bien ingeridos, almacenados y analizados, se convierten en decisiones.

1



Los datos valen cuando deciden

Las cinco V describen el reto y el pipeline de cinco etapas lo resuelve.

2



La ingesta se elige por ritmo

Lotes para lo que puede esperar; flujo para lo que no: Kinesis, Firehose, MSK, DMS o IoT Core.

3



Lago, catálogo y gobierno

S3 con zonas, Parquet, Glue para catalogar y transformar, y Lake Formation para los permisos.

4



Cada pregunta, su motor

Athena explora, Redshift sirve, Flink alerta y QuickSight muestra.

Semana 15 en cuatro ideas

Próxima semana · Semanas 16 y 17

Planificación de recuperación ante desastres y repaso general

qué pasa si falla una región: RPO, RTO, estrategias de DR y preparación del Parcial 3

Fuente: síntesis propia del curso; calendario académico 2026-2